iSingLab HKUST

Advanced AI Computing and Systems

Lecture 3: Network Evolution — From TCP/IP to AI-Native Networking

Prof. Kai Chen

THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Syllabus and Readings

iSingLab HKUST

Key Question

How should we design network systems to deliver high throughput and predictable performance, from a single host to large AI clusters?

Today's Path

[Review]	Kernel TCP/IP.
[Core]	NIC Architecture.
[Core]	Kernel-Bypass and RDMA Optimizations.
[Concept]	Scale-out and Scale-up Networks.

Readings

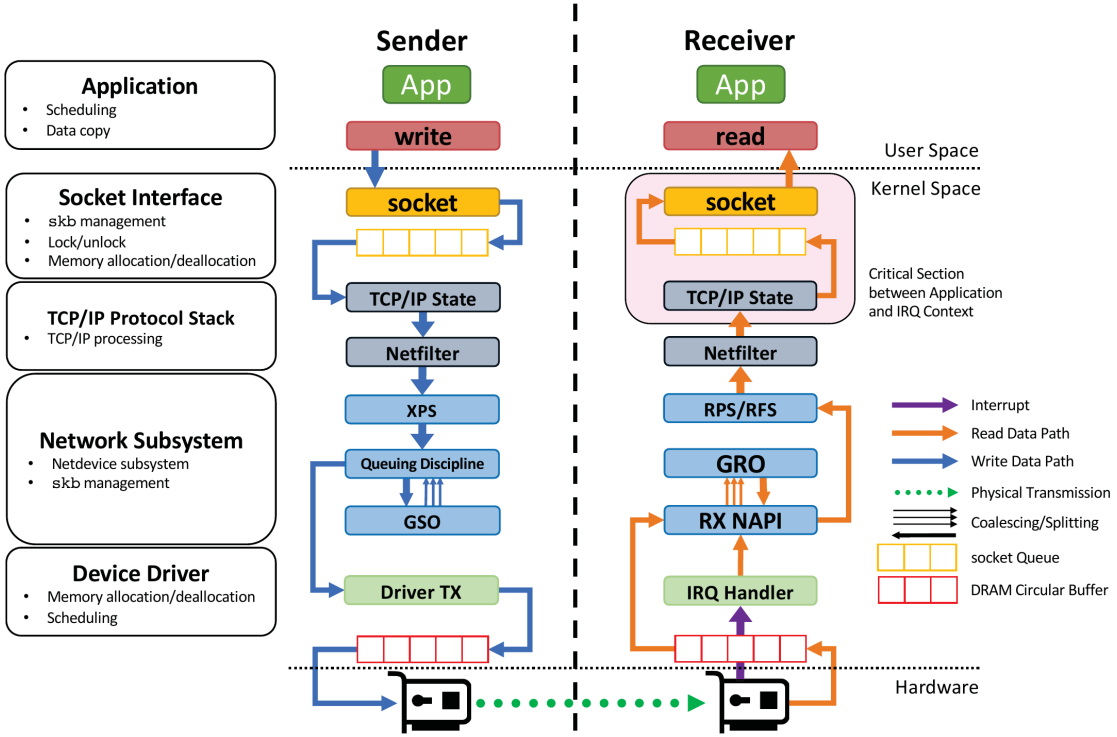
Qizhe Cai et al. Understanding Host Network Stack Overheads. SIGCOMM, 2021.

Alex Forencich et al. Corundum: An Open-Source 100-Gbps NIC. FCCM, 2020.

Anuj Kalia et al. Design Guidelines for High Performance RDMA Systems. USENIX ATC, 2016.

Kun Qian et al. Alibaba HPN: A Data Center Network for Large Language Model Training. SIGCOMM, 2024.

TCP Communication Between Two Applications



Source: Understanding Host Network Stack Overheads, SIGCOMM 2021, Fig. 1

- Application bytes cross user space, the kernel, the NIC, and the network.
- The CPU processes the stack. The NIC transfers packet data with DMA.

TCP/IP Encapsulation

Ethernet frame [IP packet [TCP segment [application bytes]]]

Layer	Data unit	Main responsibility
Application	Bytes	Application meaning and message framing
TCP	Segment	Ports, ordered delivery, and loss recovery
IP	Packet	Host addressing and routing across networks
Ethernet	Frame	Delivery over the local link

- Each layer adds information needed by its peer.
- Encapsulation adds headers. It need not copy the payload at every layer.

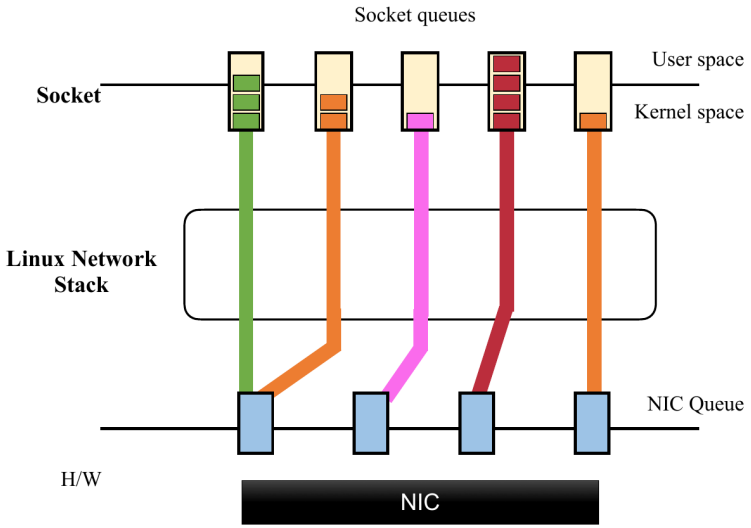
The TCP Socket Interface

Application calls

```
fd = socket(...)
connect(fd, ...)

n = send(fd, buf, len, 0)
n = recv(fd, buf, cap, 0)
```

Kernel-managed resources



(a) Linux network stack

Source: Disaggregating the Host Network Stack, SIGCOMM 2022, Fig. 1(a), logical view

- The kernel manages buffers and TCP state for each socket.
- send() accepts bytes locally. The peer application may read them later.

TCP Byte-Stream Service

Sender writes	Possible receiver reads
send("AB") **send("CD")**	recv(): "A" recv(): "BCD"

Read boundaries can differ from write boundaries

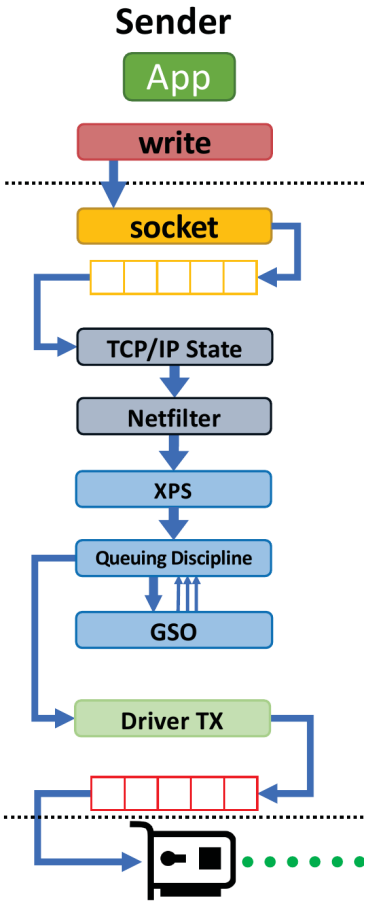
Sequence numbers and ACKs track delivery.
Retransmission repairs loss.

- TCP delivers a reliable, ordered byte stream.
- Flow control protects receiver buffers. Congestion control limits network load.

The Kernel Transmit Path

1. Copy application bytes into a kernel buffer.
2. Process TCP state and protocol headers.
3. Queue packet work and pass it to the driver.
4. Let the NIC fetch packet data with DMA.
5. Retain unacknowledged bytes for retransmission.

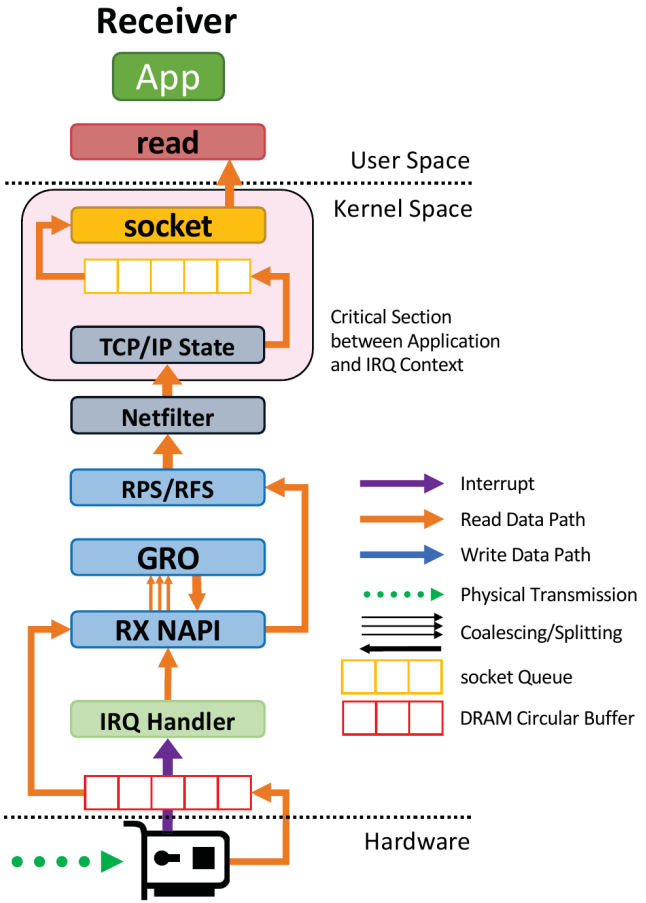
The user-to-kernel boundary copies the payload in this ordinary path.



The Kernel Receive Path

1. The NIC DMAs data into a kernel buffer.
2. An interrupt schedules receive processing.
3. NAPI, GRO, and TCP/IP process the data.
4. TCP makes ordered bytes available to the socket.
5. recv() copies bytes into the application buffer.

Packet arrival and application consumption are different events.



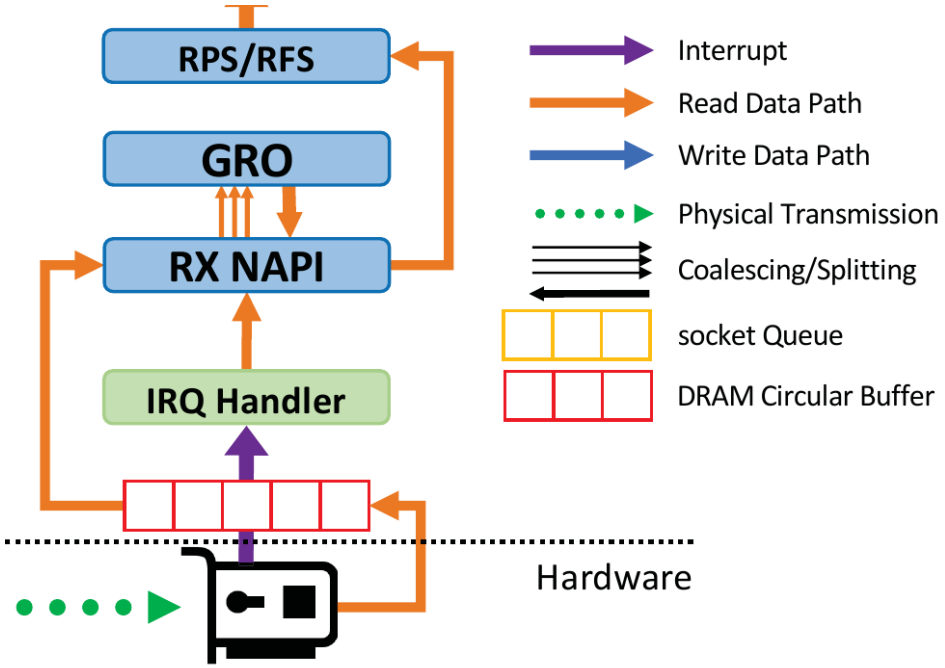
Interrupts and NAPI

Receive processing

1. An IRQ schedules NAPI.

2. Poll work in batches, up to the receive budget.

3. When work is drained, complete NAPI and re-enable interrupts.



- One interrupt can lead to processing multiple received packets.
- Notification starts processing. DMA moves the packet bytes.

TSO and GRO

Mechanism	Description
TSO (Tx)	TCP Segmentation Offload. Offloads the segmentation of "big" packets (up to 64KB) into frames to the NIC.
GRO (Rx)	Generic Receive Offload. Aggregates MTU-sized frames into a "big" packet (up to 64KB) before passing them to the TCP/IP layer.

GSO provides software segmentation. TSO delegates it to the NIC.
GRO combines received segments before TCP processing.

- Larger processing units amortize protocol and buffer-management work.
- Wire packets still obey the MTU. Receive aggregation happens at the host.

Packet Processing at 100 Gb/s

100 Gb/s = 12.5 GB/s nominal link rate

Ethernet frame	Wire occupancy	Packet rate	Cycles/frame
64 B	84 byte-times	148.8 Mpps	20.2
1518 B	1538 byte-times	8.13 Mpps	369.1

Add 8 B preamble/SFD and 12 byte-times of inter-frame gap.
Cycles/frame = 3 GHz / packet rate, for one idealized CPU core.

Calculated at 100 Gb/s. Nominal MAC rate, before payload and processing overheads.

- Small packets leave very little CPU work per frame.
- Actual throughput also depends on memory, queueing, and offloads.

NIC Architecture

Lecture 3: Network Evolution
From Kernel Stack to 100Gbps RDMA

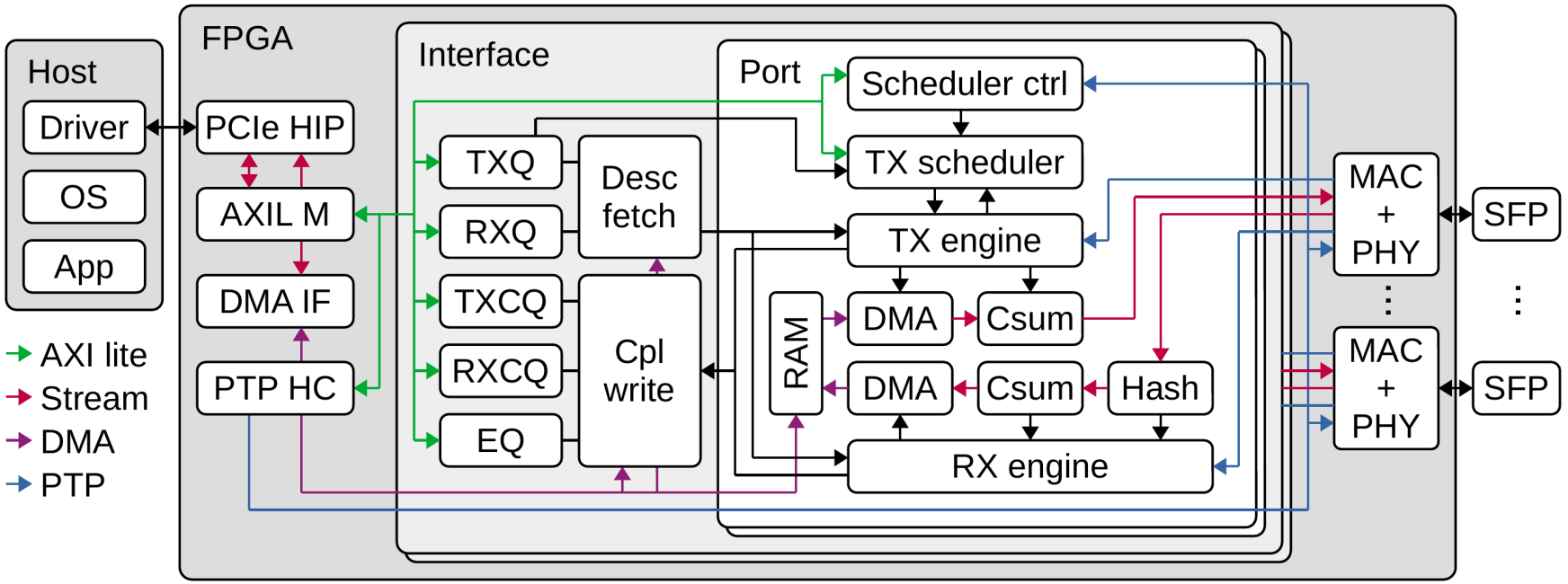
Queues and descriptors
DMA and completion
Hardware pipelines



THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

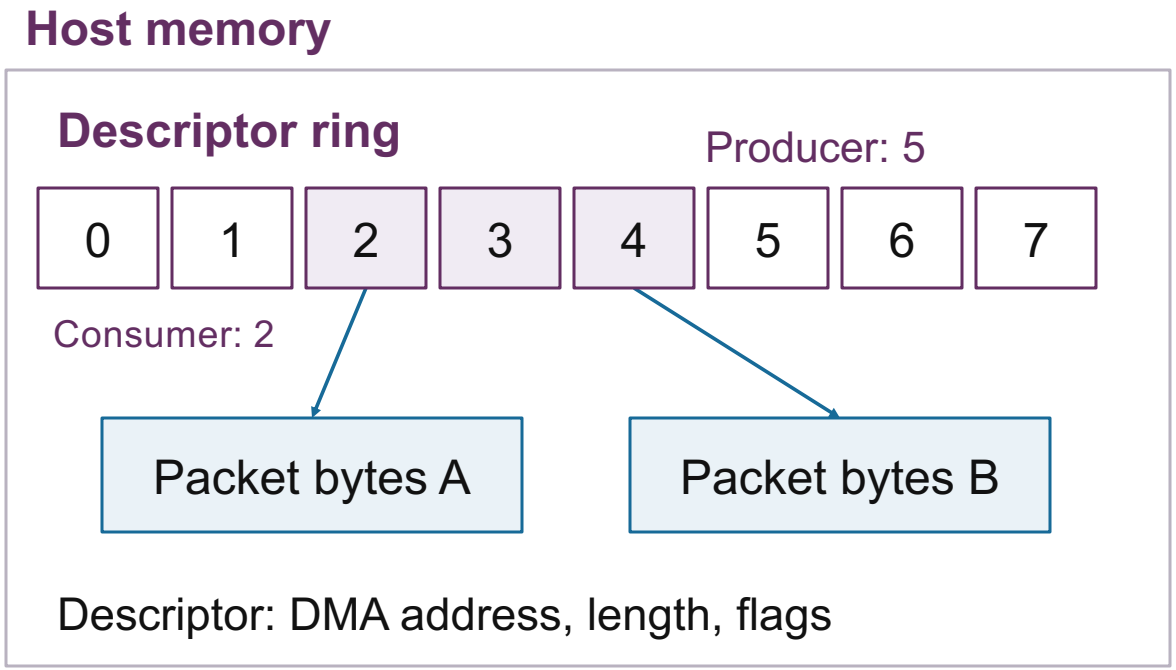
Inside a Modern NIC



Source: Corundum, FCCM 2020, Fig. 1

- PCIe and DMA connect host memory to the NIC packet pipeline.
- Queue managers track work. Engines and the scheduler execute it.

Descriptors Point to Packet Buffers



NIC queue state

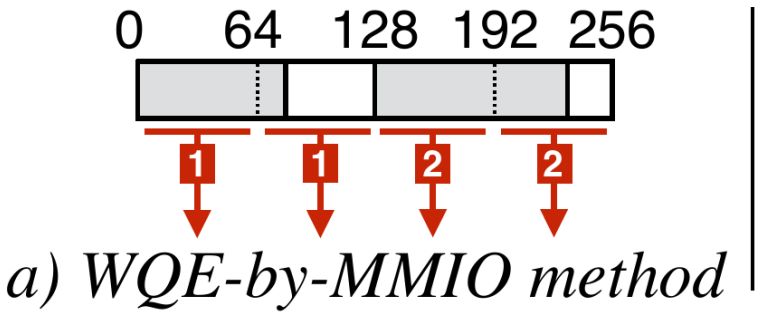
Ring base
Ring size
Producer progress
Consumer progress

After slot 7, the index wraps to slot 0.

- Descriptors identify work and point to packet buffers.
- Producer and consumer progress control when slots and buffers can be reused.

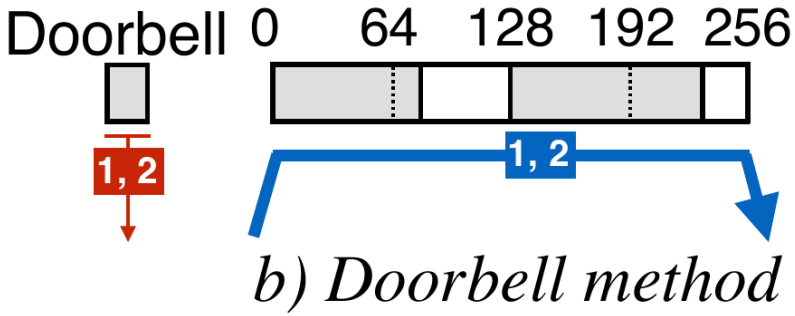
Doorbells Notify; DMA Moves Work and Data

CPU pushes descriptors



MMIO writes carry the work descriptors.

CPU notifies, NIC fetches



Blue arrow above: descriptor DMA

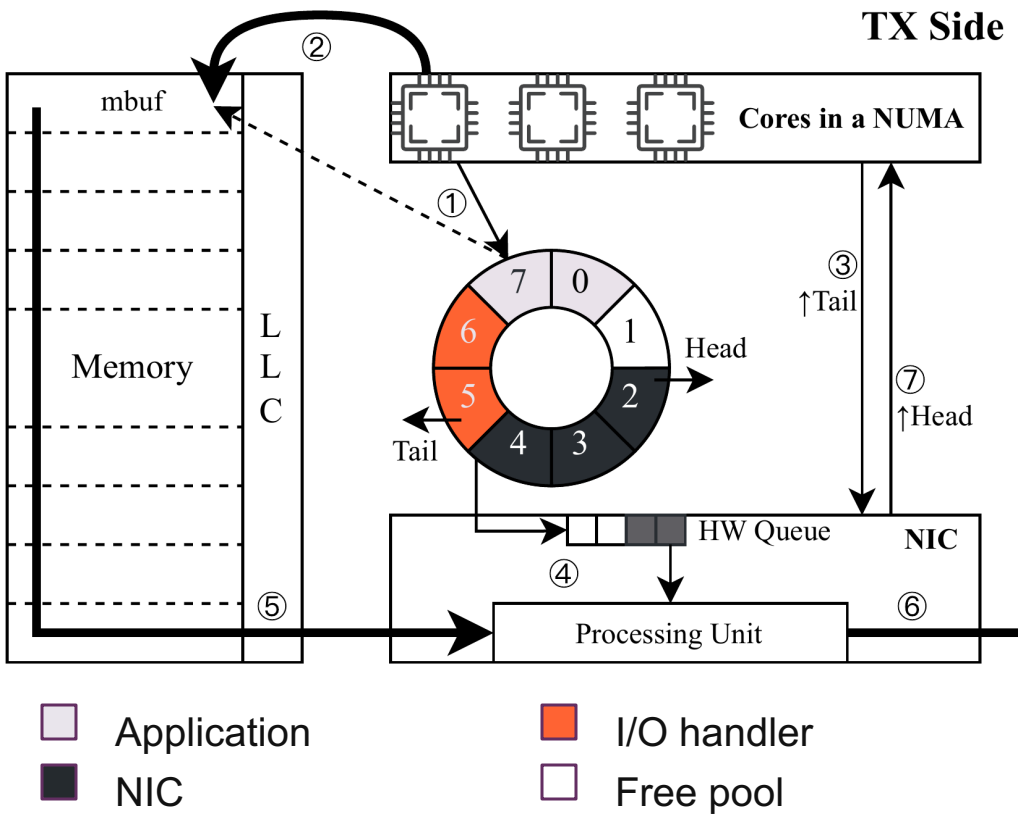


Separate payload DMA read

Source: RDMA Guidelines, ATC 2016, Fig. 4. WQE means work descriptor in this example.

- MMIO accesses device registers or mapped device memory.
- A doorbell notification can trigger larger DMA transfers.

Transmit: Prepare, Publish, Fetch, Send



Ring colors show logical buffer ownership.

1–2 Prepare and fill a buffer.

3 Publish descriptors and notify the NIC.

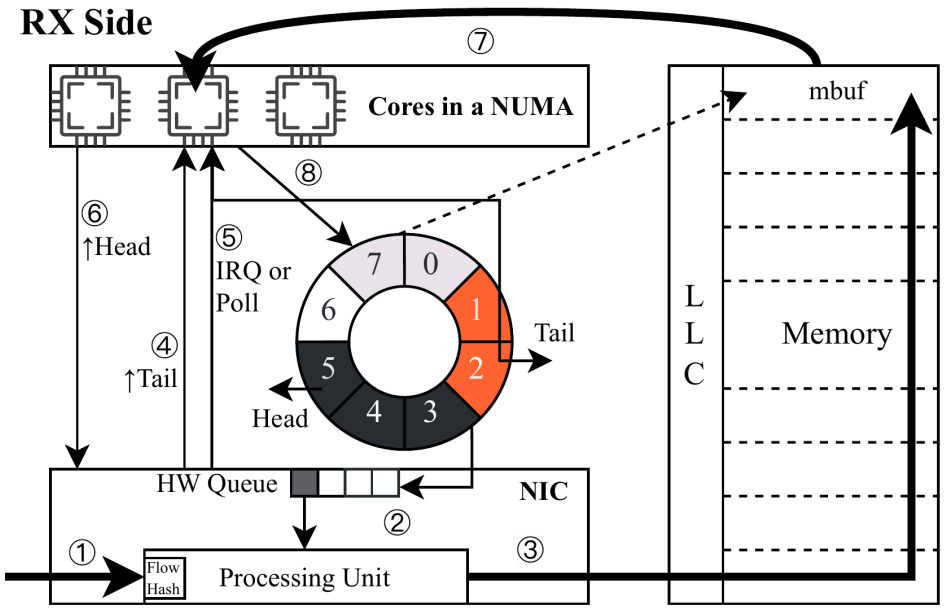
4–5 DMA-read descriptors and packet bytes.

6 Transmit through MAC/PHY.

7 Observe completion and reclaim resources.

Receive: Post Buffers Before Packets Arrive

Software posts available receive buffers before packets arrive.



Ring colors show logical buffer ownership.

1–2 Classify the packet and select a queue.

3 DMA-write bytes into a posted buffer.

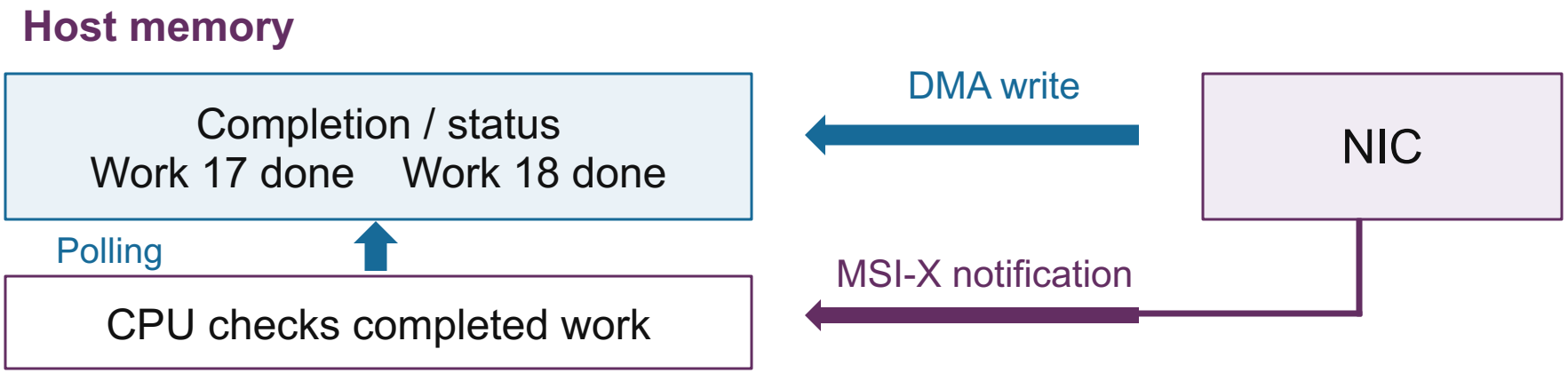
4–5 Publish completion. Software notices it.

6 Deliver packets and refill RX buffers.

7–8 Process, then release or forward.

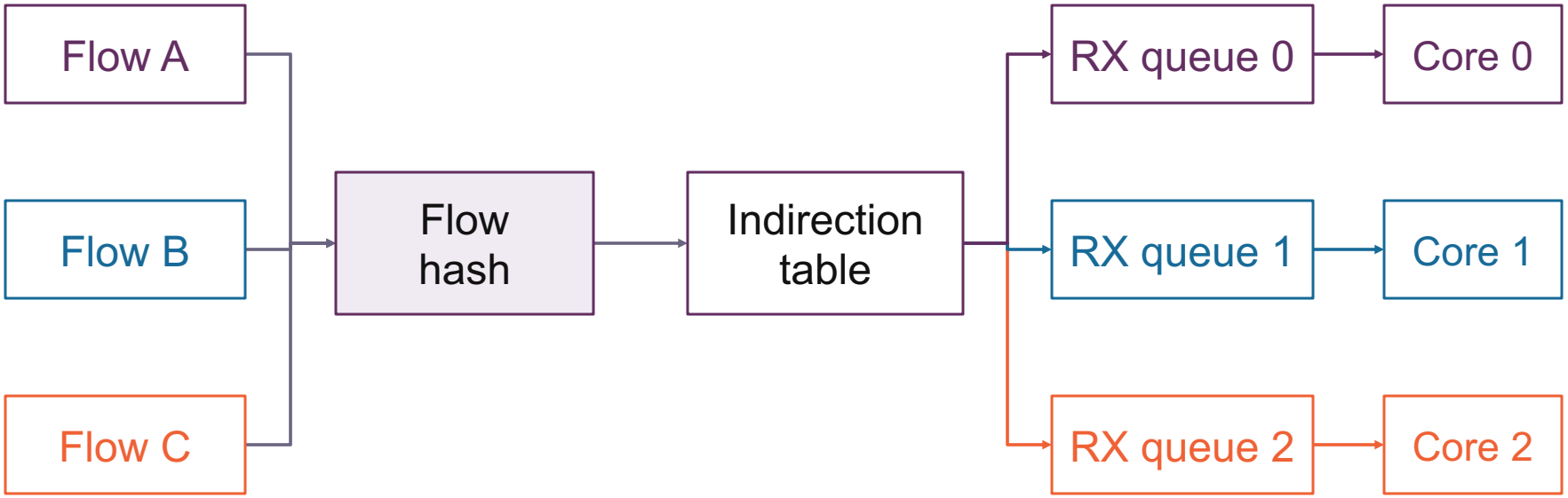
Free pool: allocate / recycle

Completion Records Work; Interrupts Notify Software



Queue / ring	Information it carries
Descriptor ring	Work to perform or available RX buffers
Completion queue	Results of completed work
Event queue	Queue events, as in Corundum
Software handoff ring	Buffer references between software stages

RSS Maps Flows to Receive Queues



- A stable RSS mapping keeps one flow on one receive queue.
- CPU affinity maps receive queues to processing cores.

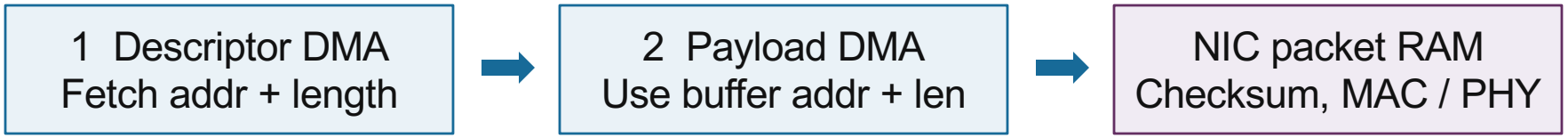
Queue State Drives DMA Requests

NIC queue-state RAM (illustrative values)

Queue	Ring base	Slots	Producer (head)	Consumer (tail)
TX 0	0x1000	8	5 (software)	2 (NIC)
RX 0	0x2000	8	6 (software)	4 (NIC)
CQ 0	0x3000	8	3 (NIC)	1 (software)

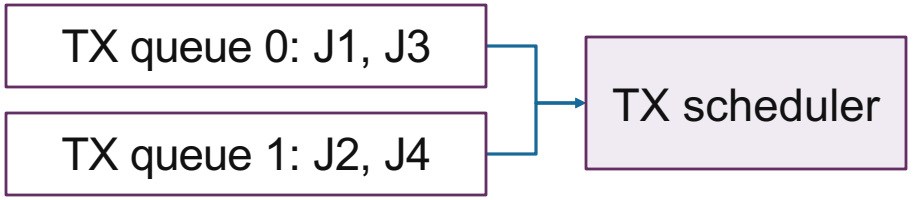
TX descriptor address = base + (reserved index mod size) × stride

Reserved slot 2: 0x1000 + (2 mod 8) × 16 B = 0x1020

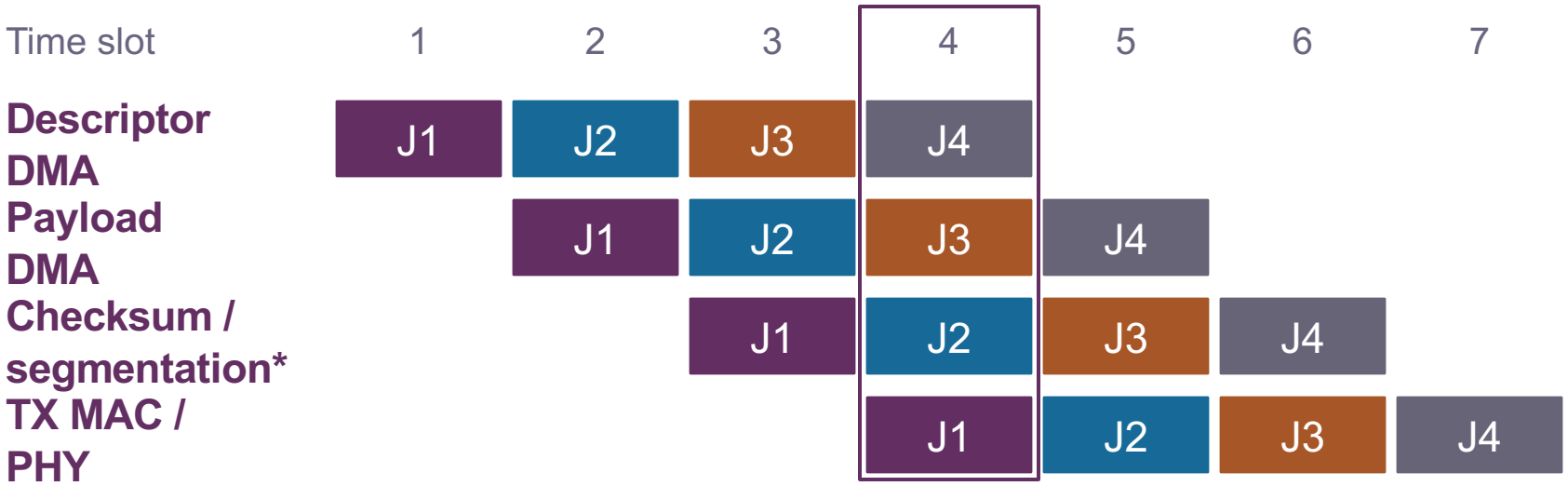


TX/RX descriptor queues: software produces, NIC consumes.
Completion queues reverse the producer and consumer roles.
RX uses its descriptor to DMA-write received payload into host memory.

Keeping the NIC Pipeline Busy



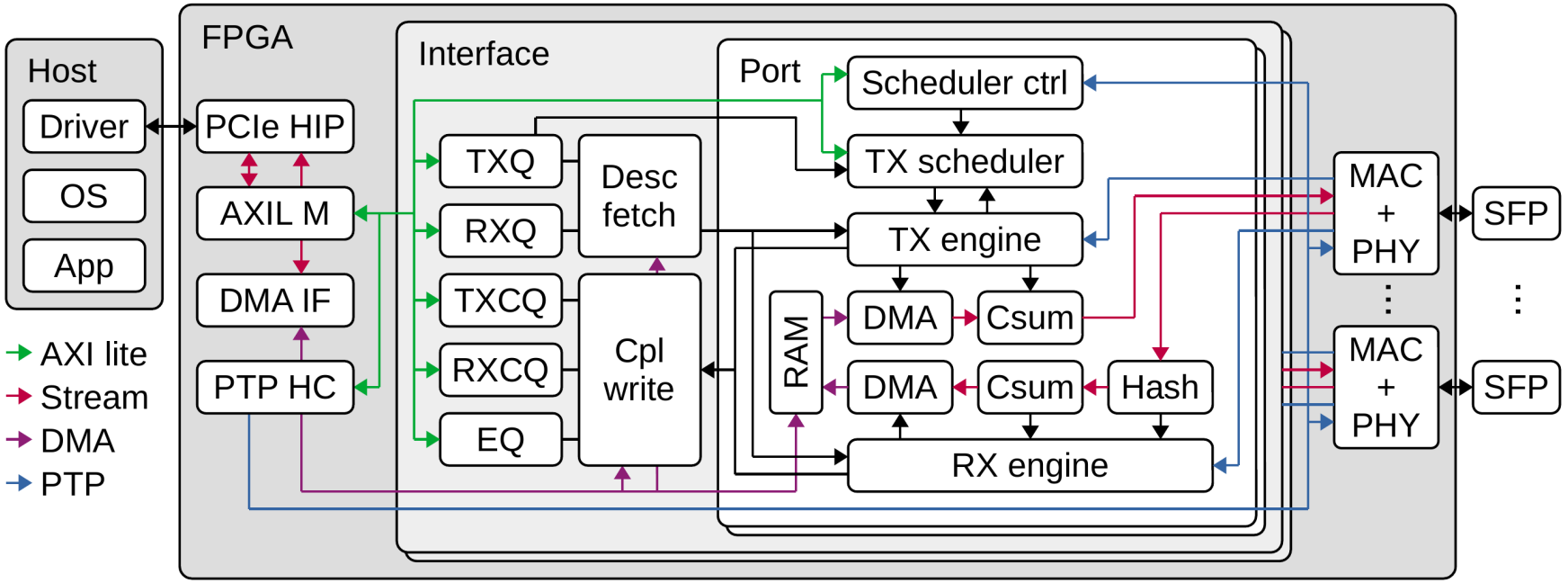
Multiple queues supply work. Each queue can also keep several jobs in flight.



At slot 4, all four stages work concurrently.

100 Gb/s, 64 B frames: 148.8 Mpps = 6.72 ns/frame (A, p. 9).

Putting It All Together



Source: Corundum, FCCM 2020, Fig. 1 and §II.A–F

1. Control/time: PCIe HIP, AXIL M, PTP HC, queue state.
2. TX work: TXQ, Desc fetch, scheduler/control, TX engine.
3. TX bytes: DMA IF, RAM, port DMA, Csum, MAC/PHY, SFP.
4. RX: MAC/PHY, Hash, Csum, RX engine, RXQ, host buffer.
5. Results: Cpl write, TXCQ/RXCQ, EQ, PCIe notification.

Kernel Bypass and Partial Offload

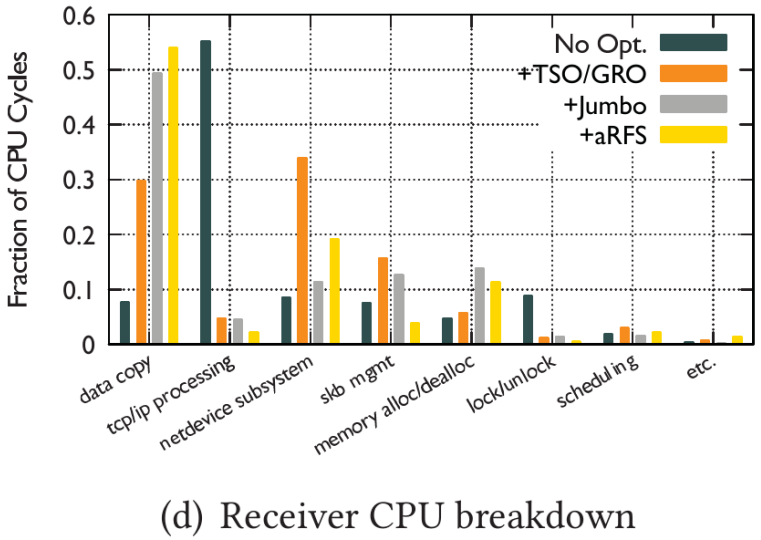
Lecture 3: Network Evolution
From Kernel Stack to 100Gbps RDMA

Userspace packet I/O
Programmable packet paths
Selective protocol offload

THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

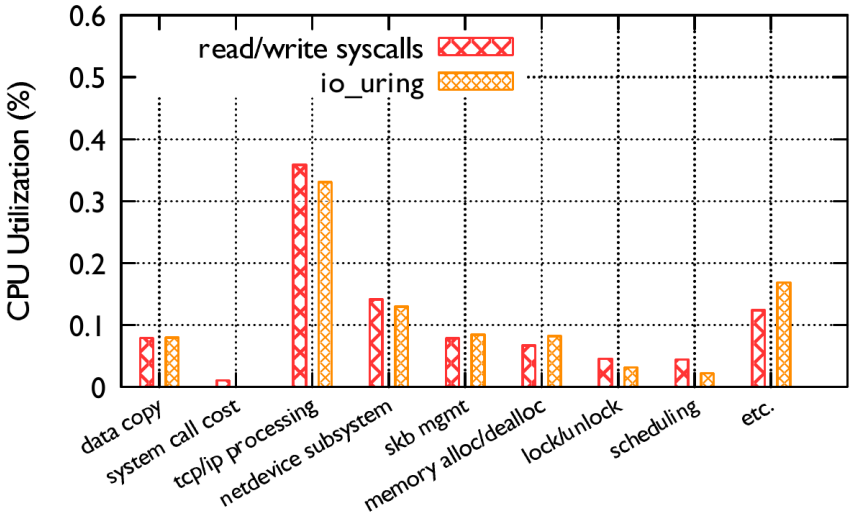
Where Kernel TCP/IP Spends CPU Cycles

Long TCP flow: receiver



(d) Receiver CPU breakdown

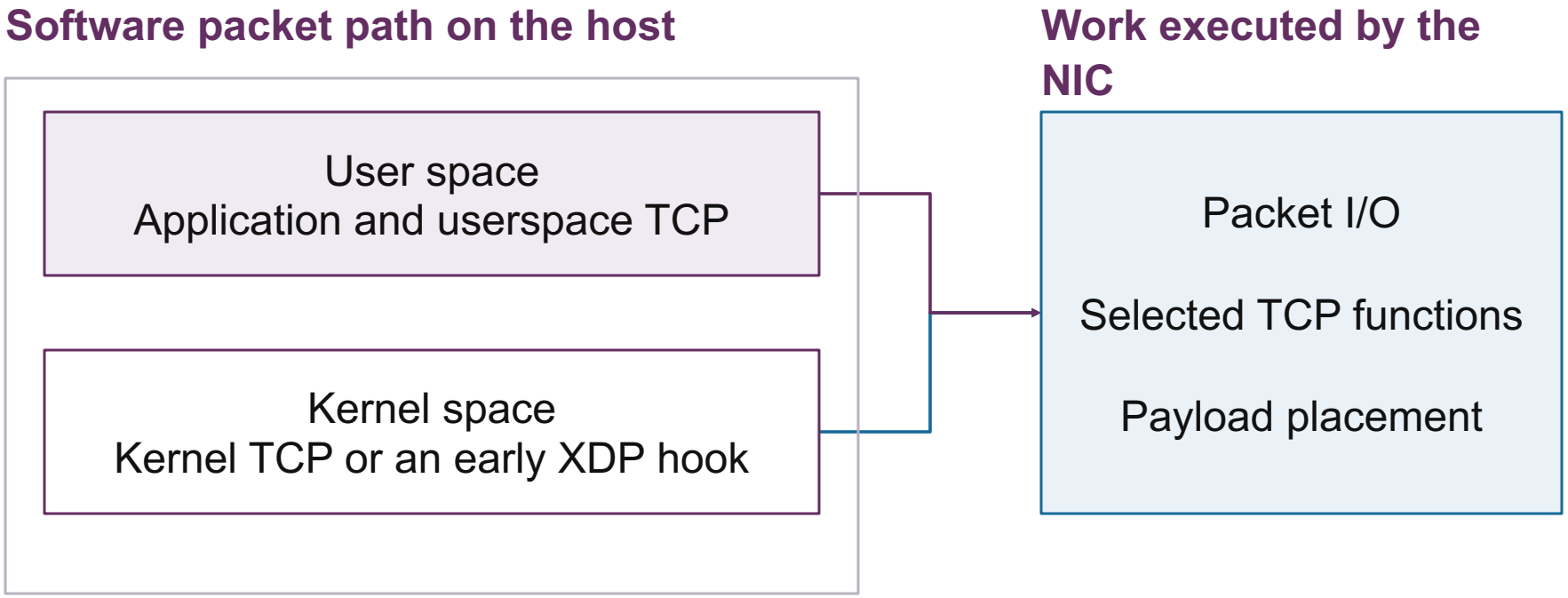
4 KB RPCs: sender



Source: Understanding Host Network Stack Overheads, SIGCOMM 2021

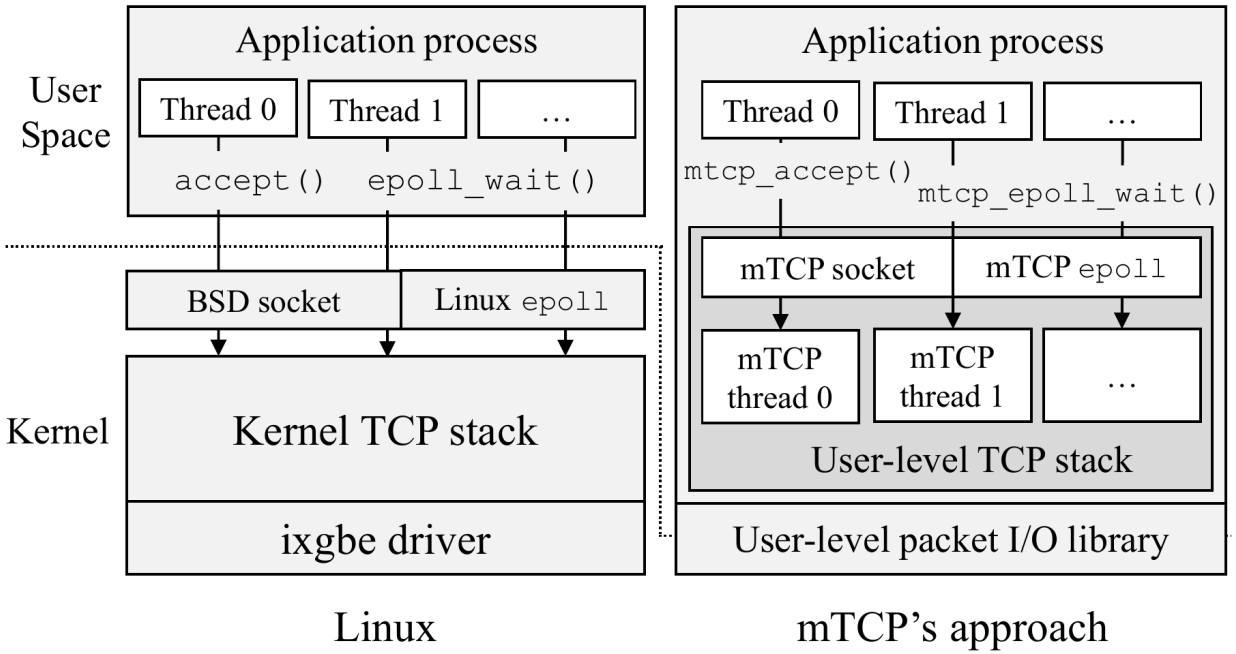
- Host CPU cycles go to protocol handling, copying, and buffer management.
- These are separate experiments with different workloads and platforms.

Software Paths and Hardware Offload



- Kernel bypass changes the software fast path.
- Offload changes which functions the NIC executes. The choices can combine.

Let Userspace Drive the Queues

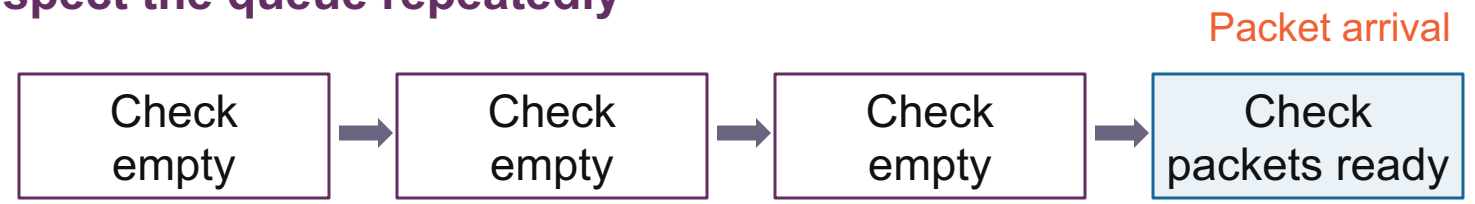


Source: mTCP, NSDI 2014, Fig. 3. The original implementation uses PSIO.

- The kernel sets up controlled device and memory access.
- The userspace fast path still runs TCP logic on host CPU cores.

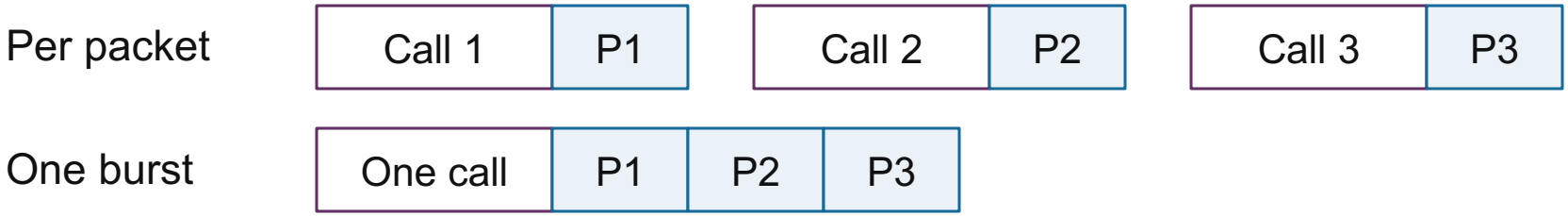
Polling and Batching Have Trade-offs

Polling: inspect the queue repeatedly



Empty checks consume CPU cycles.

Batching: process packets already available



- Polling avoids waiting for a notification, at the cost of idle CPU work.
- Burst processing amortizes overhead. Waiting to fill a batch adds delay.

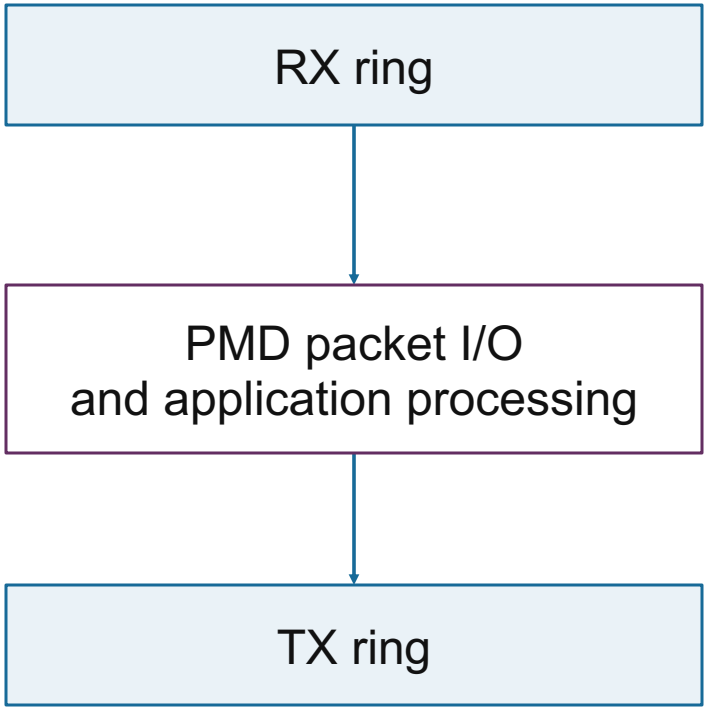
DPDK Provides Packet I/O

Application packet loop

```
n = rx_burst(rxq, pkts)

process(pkts, n)

accepted = tx_burst(txq, pkts, n)
```



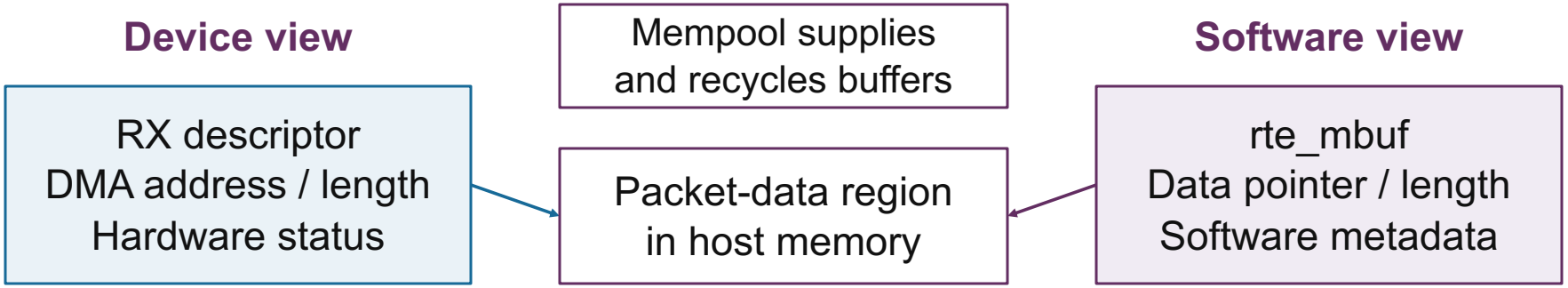
Optional software TCP stack
inside process()

mbuf: metadata and packet-data reference
Mempool: buffer allocation and recycling

Packet I/O provides packets. An upper layer provides TCP semantics.

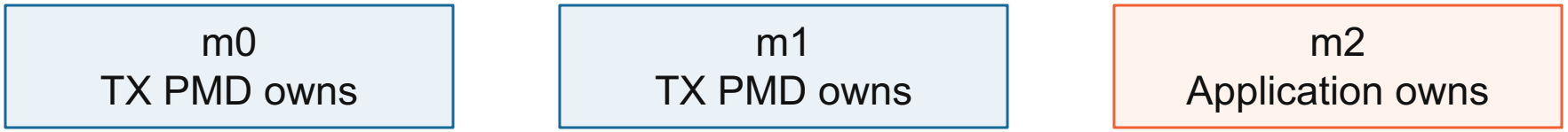
DPDK Buffers and Queue Ownership

Two views of the same packet-data region



rx_burst() returns mbuf pointers to the application.

TX example: offered m0, m1, m2. Accepted = 2.



Accepted buffers: PMD reclaims after device use.

Retry or free.

XDP Processes Packets Before the Stack

Native driver XDP

PASS

Continue through kernel TCP/IP.

DROP

Stop processing this packet early.

TX

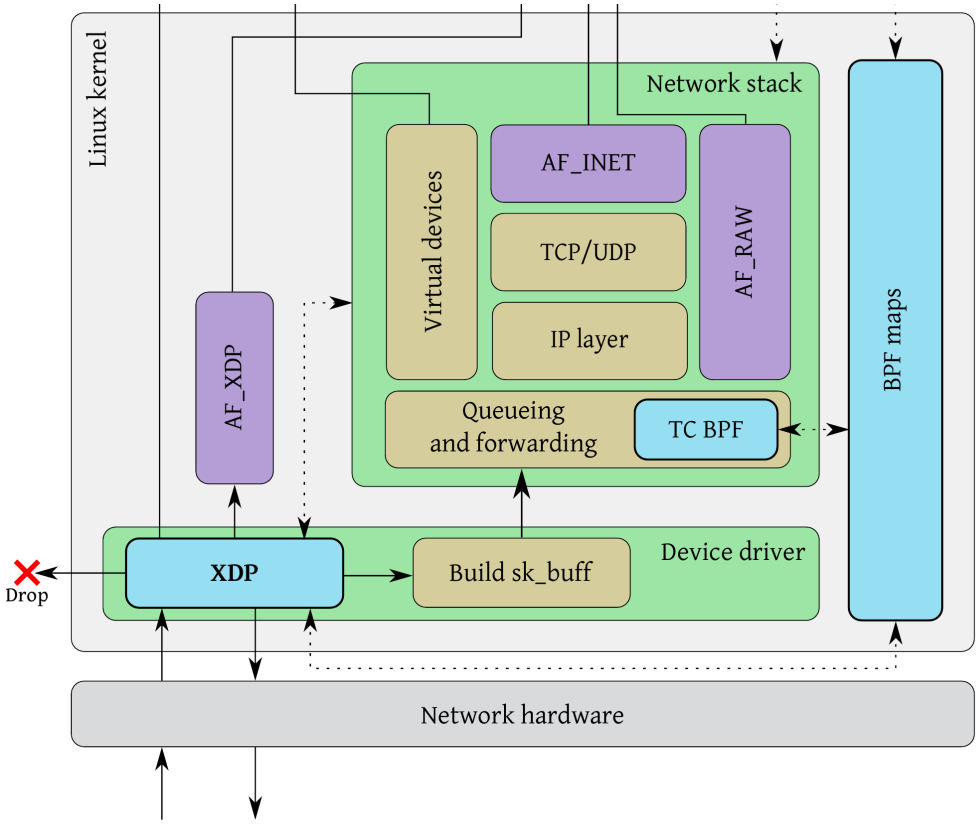
Send back through the same interface.

REDIRECT

Send to another interface, CPU, or AF_XDP.

eBPF = program
XDP = hook

Upper arrows continue to userspace applications and control logic.



Sources: The eXpress Data Path, CoNEXT 2018, Figs. 1–2; Linux AF_XDP

XDP Programs, Maps, and Verification

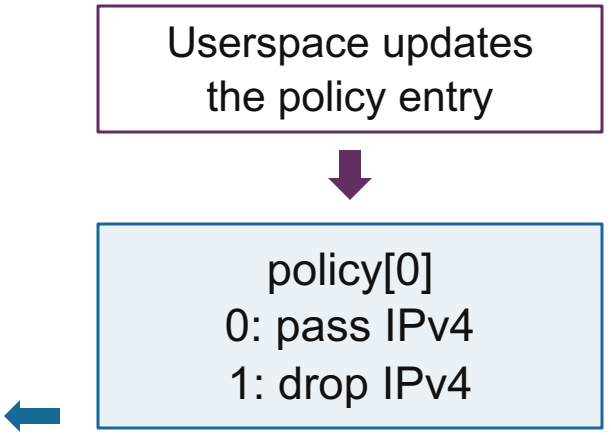
Program loading



Per-packet execution

```
eth = (void *)(long)ctx->data;
end = (void *)(long)ctx->data_end;
if ((void *)(eth + 1) > end) return XDP_DROP;
if (eth->h_proto != bpf_htons(ETH_P_IP))
    return XDP_PASS;
key = 0;
mode = bpf_map_lookup_elem(&policy, &key);
return mode && *mode ? XDP_DROP : XDP_PASS;
```

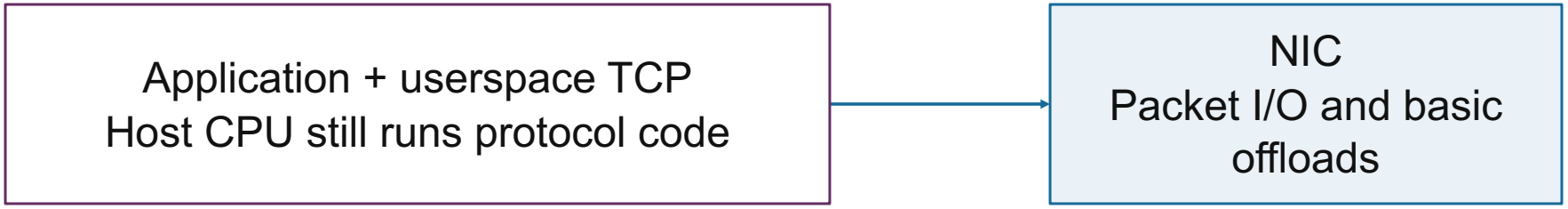
C-like excerpt: declarations and map setup omitted.



Map updates change policy without reloading the program.

Beyond Kernel Bypass: Offload TCP Work

After kernel bypass

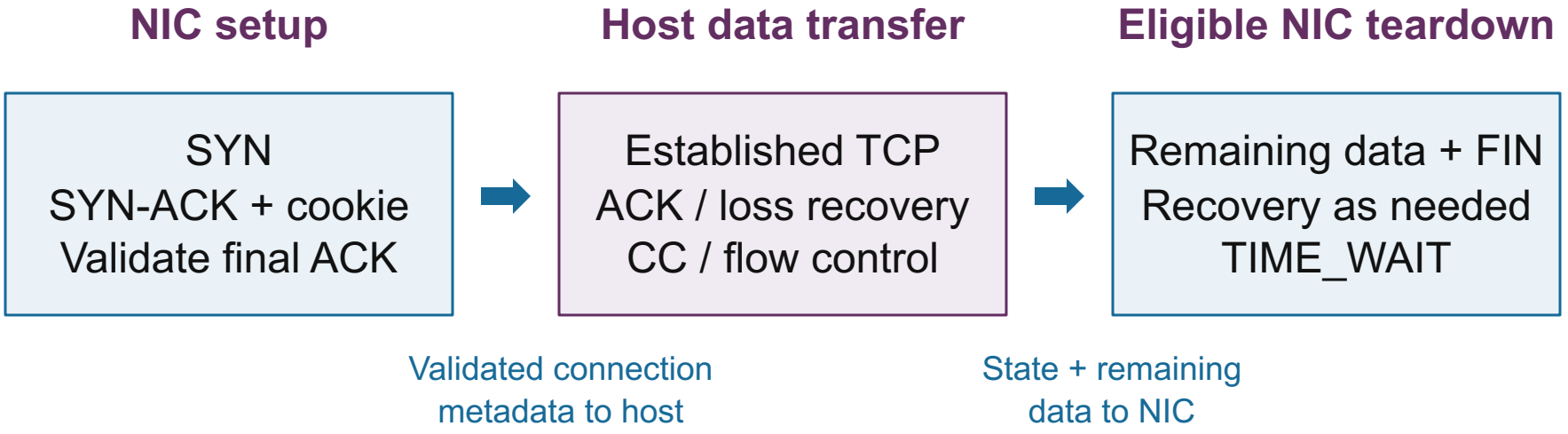


Additional functions can move with their state

Offload candidate	Work the NIC may execute
Connection management	SYN / FIN lifecycle
Established TCP	SEQ, ACK, windows, fast recovery
Payload movement	Split / merge, direct placement

Each design chooses a function boundary and a state owner.

Offloading Connection Management



One owner updates connection state at a time.

Teardown offload requires an eligible close and state handoff.

Source: AccelTCP, NSDI 2020, Fig. 5(a–b) and §§3–4. Simplified lifecycle.

Offloading the Established TCP Fast Path

Host / control plane

Connection management
Congestion policy
Timeout recovery triggers

Application + libTOE
Socket payload buffers
in host memory

Policy / control

Payload DMA

Programmable NIC pipeline

Preprocess
Parse + lookup

Protocol
SEQ / ACK / window
Fast retransmit

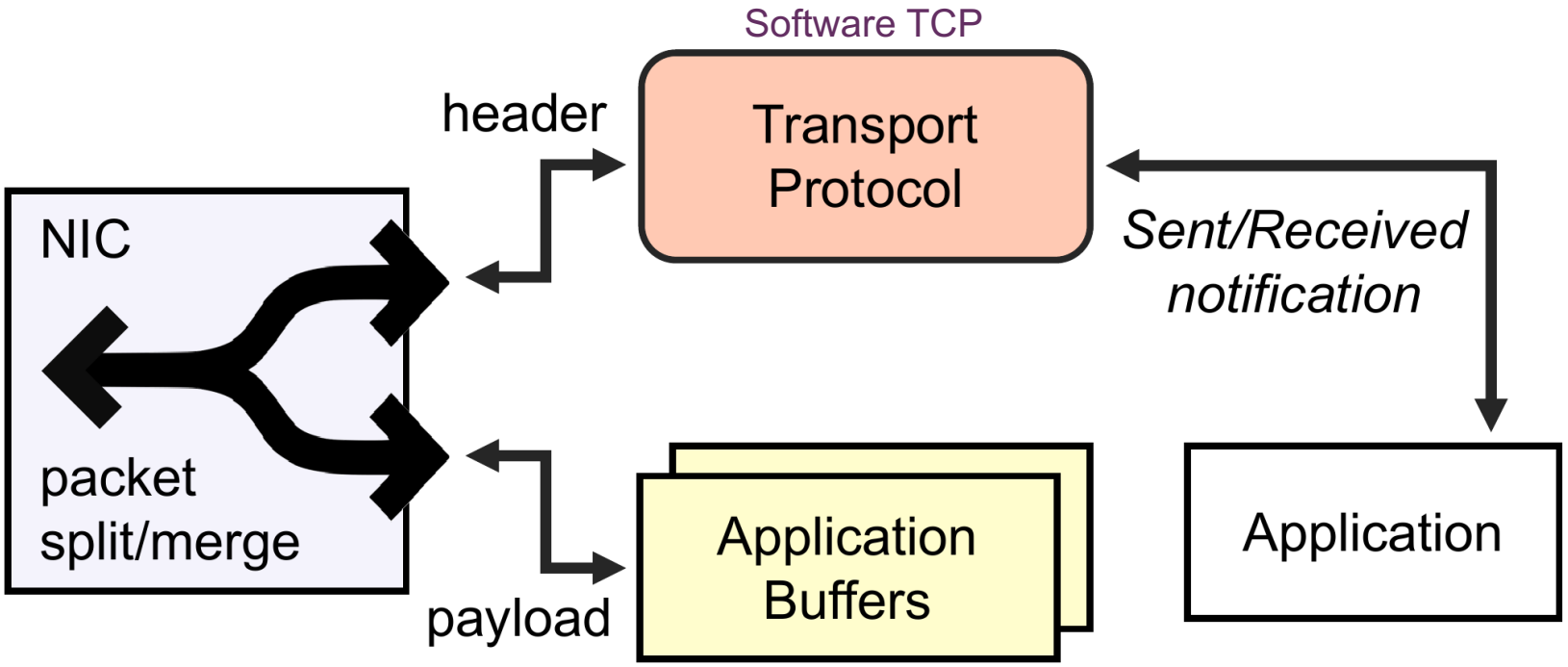
Postprocess
DMA
Notify application

Protocol updates remain atomic per connection.

NIC flow scheduling enforces the software congestion policy.

Source: FlexTOE, NSDI 2022, Figs. 2–3 and §3.1. Simplified host/NIC placement.

Separate Protocol Control from Payload Movement



Source: High-throughput and Flexible Host Networking, OSDI 2024, Fig. 3

- Software TCP keeps CC, loss recovery, and delivery decisions programmable.
- The NIC splits headers from payload and moves bytes to application buffers.

RDMA

Lecture 3: Network Evolution
From Kernel Stack to 100Gbps RDMA

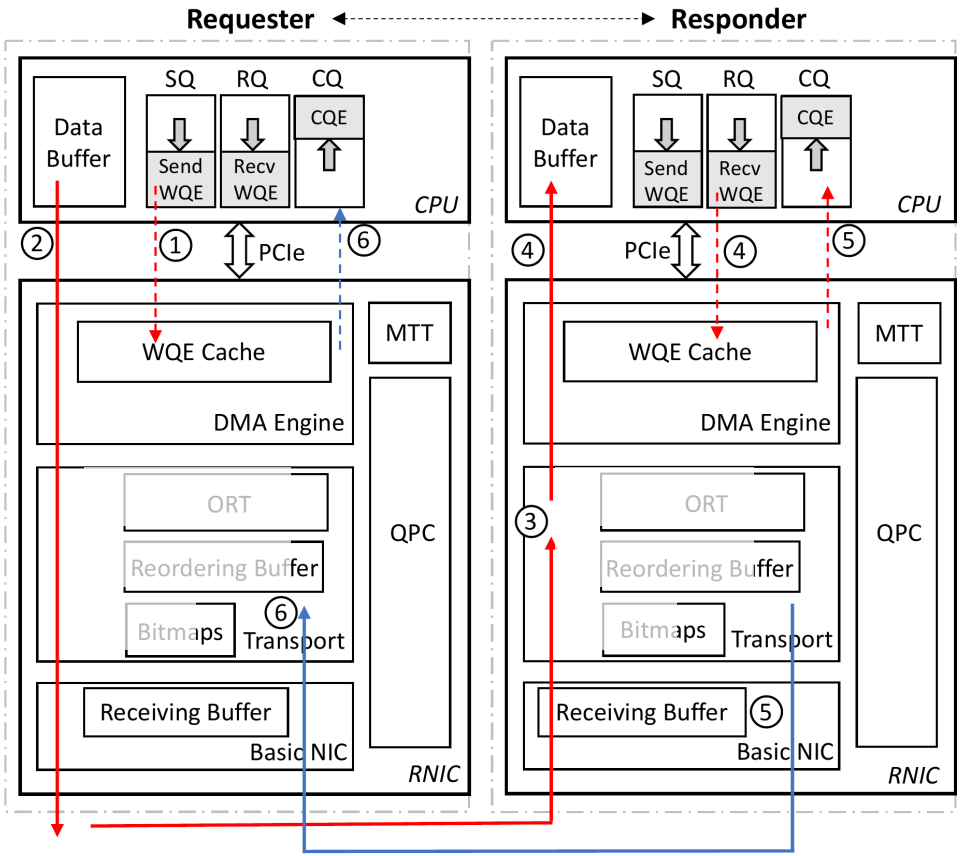
Operations and workflows
Transport in hardware
InfiniBand and Ethernet



THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

RDMA: Hardware-Based Transport



Host

Application and setup
Work submission
Completion handling

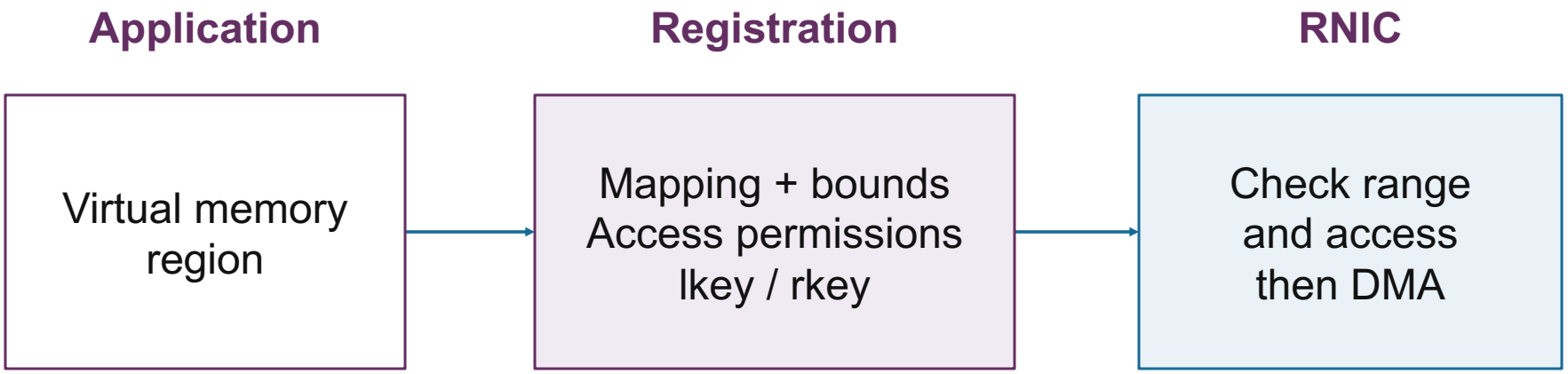
RNIC

Transport processing
DMA and direct placement
Address translation

Buffers and queues still use host memory.

Source: SRNIC, NSDI 2023, Fig. 3.

Registered Memory and Access Protection

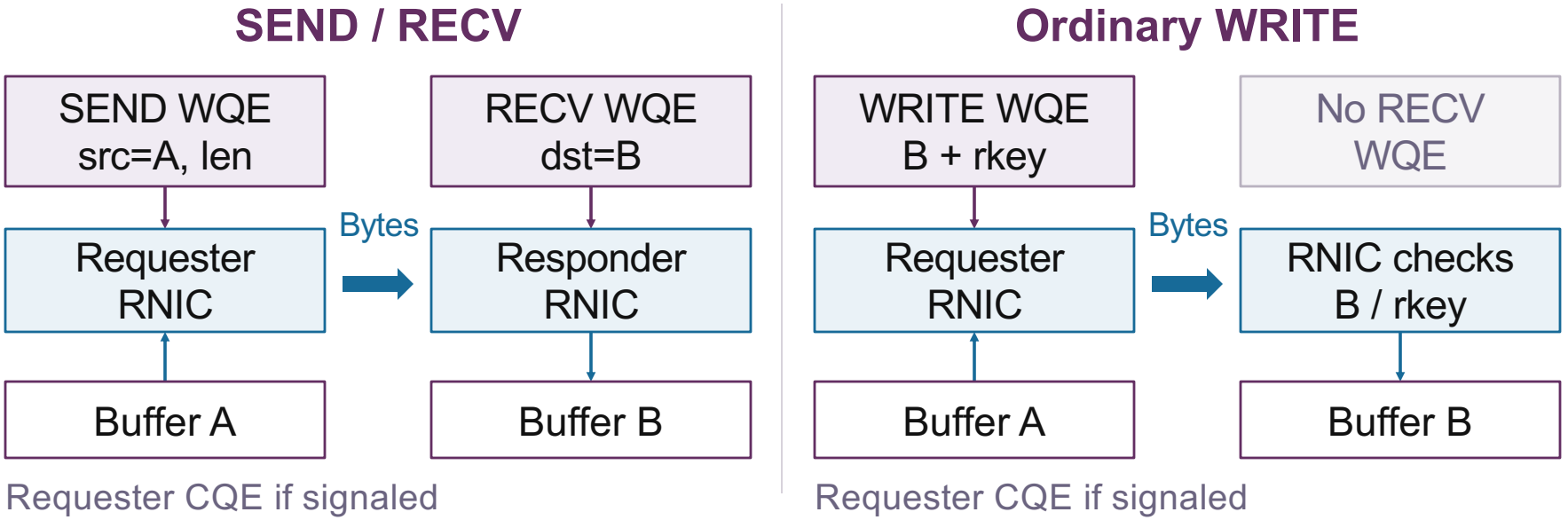


Request	Result
Valid range + permitted access	DMA allowed
Outside bounds or denied access	Rejected

- Registration follows the resource lifetime, rather than each packet.

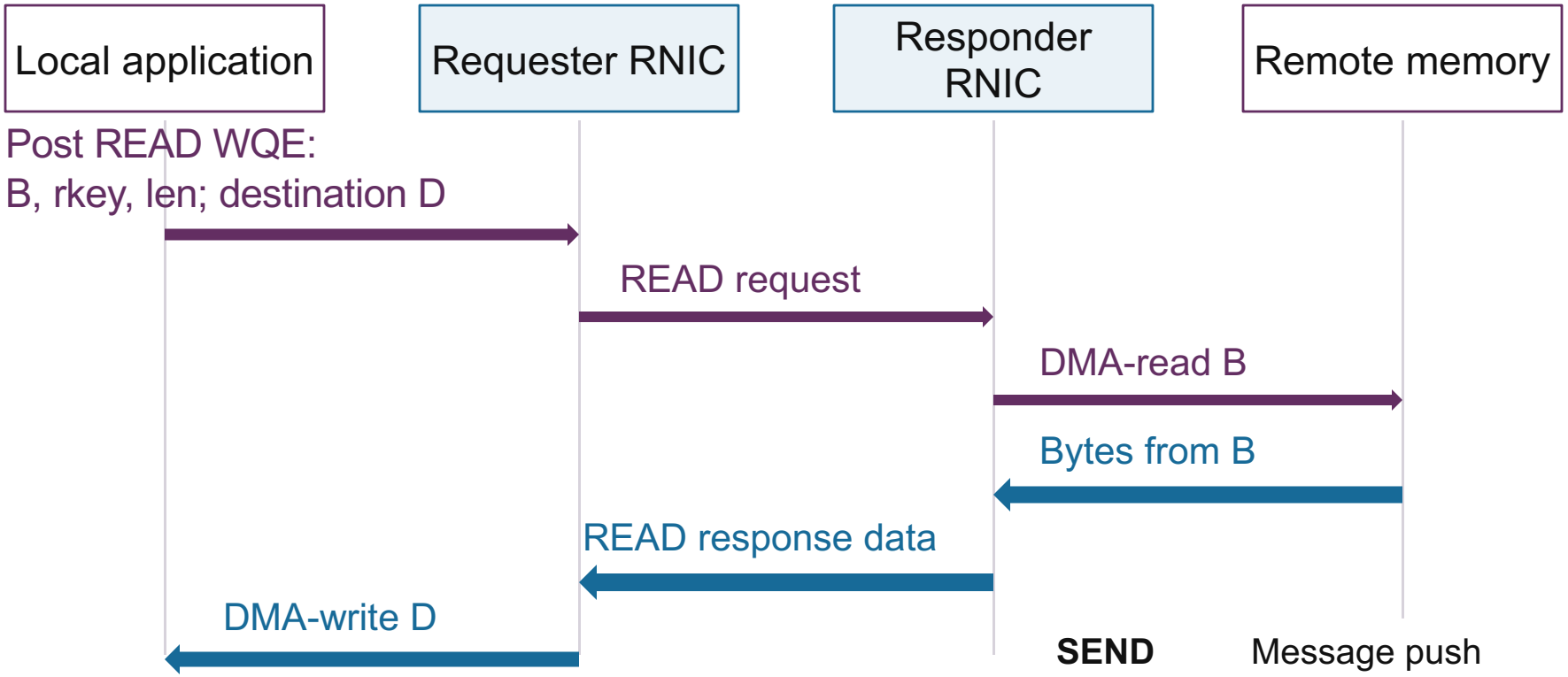
  Keys authorize access to the region. They do not encrypt its bytes.

SEND vs WRITE: Who Chooses the Remote Buffer?



	SEND	WRITE
Remote buffer	Receiver via RECV	Requester via WRITE
Remote receive WQE	Required	Unused
Remote receive CQE	RECV completion	None for ordinary WRITE

RDMA READ: Request and Response

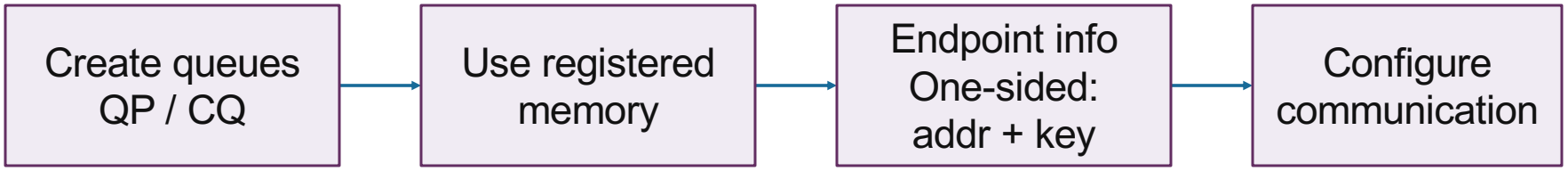


SEND	Message push
WRITE	Addressed push
READ	Addressed pull

If signaled, local CQE follows placement.
Remote app granted access earlier.

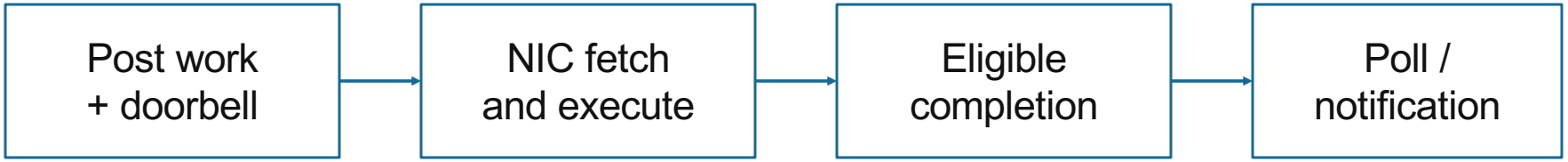
RDMA Control Path

Setup / resource changes



Create, update and release resources over their lifetime.

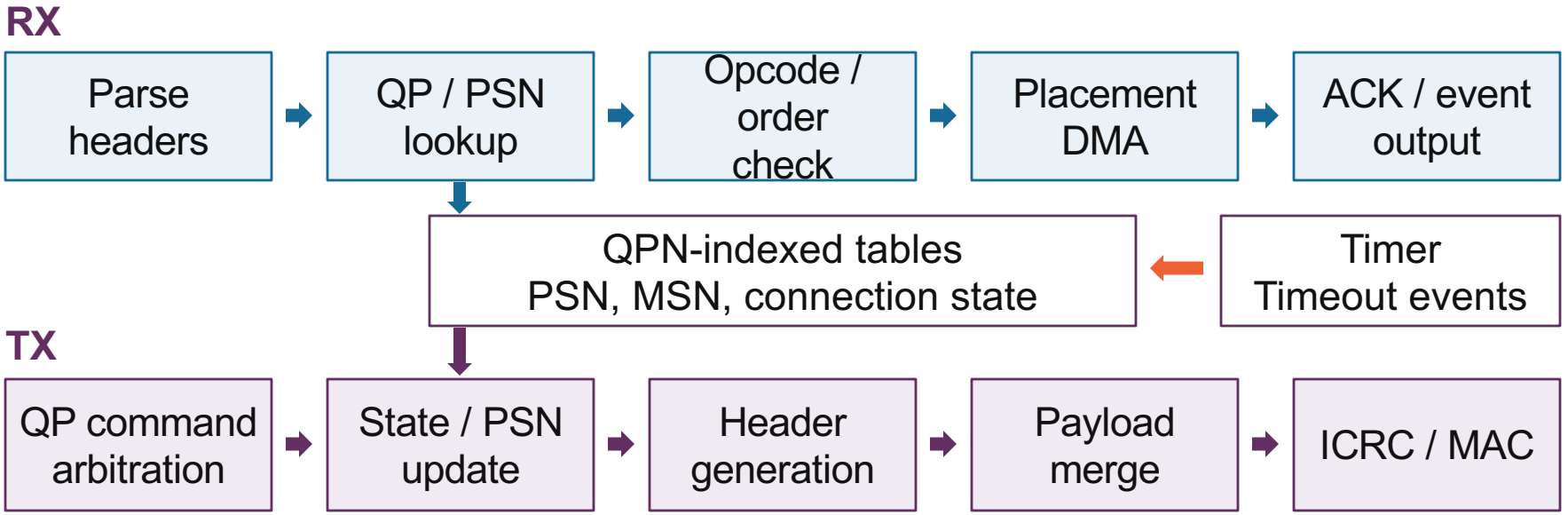
Repeated data path



- Submitting work and polling completions are part of the data path.

Inside the RDMA Transport Engine

RX and TX pipelines share indexed transport state.



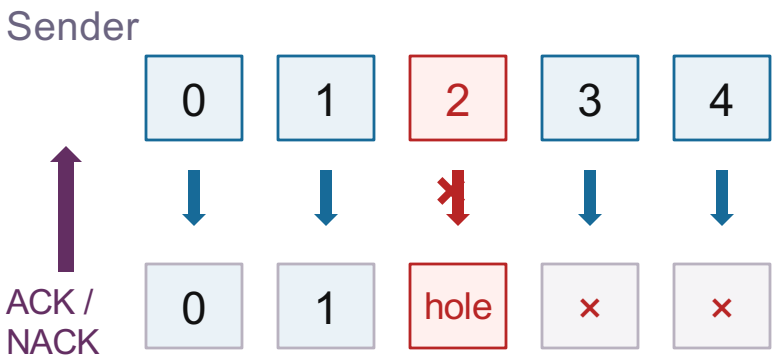
Payload: wide streaming path

Metadata / events: narrow path with ordered state updates

Recovery Ordering Rate control PFC at MAC / link

Go-Back-N and Selective Retransmission

Go-Back-N



Receiver discards 3 and 4.

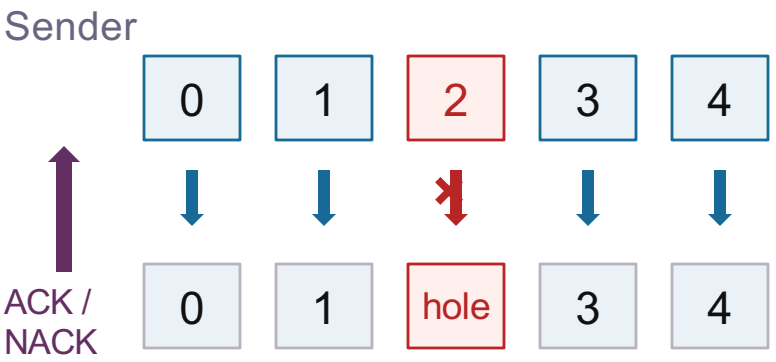
NACK: expected PSN = 2

Retransmit



3 packets sent again

Selective retransmission (IRN)



Receiver retains 3 and 4.

NACK: expected = 2, received = 3 / 4

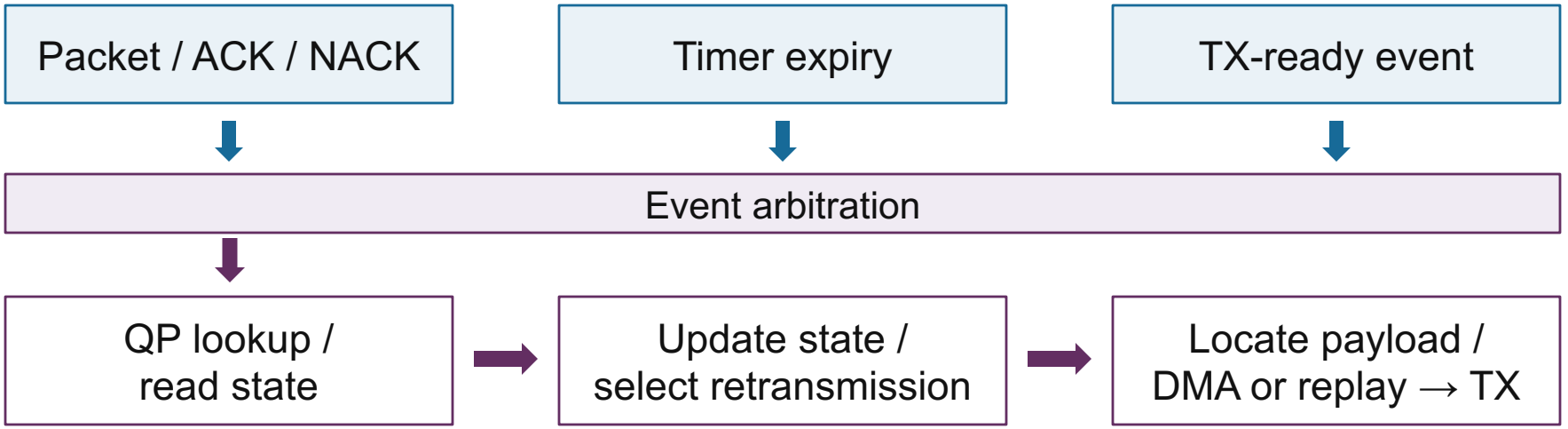
Retransmit



1 packet + receipt state

Selective recovery saves retransmissions and tracks more state.

Reliability Hardware: State, Timers, and Replay



State	Purpose
Outstanding range	First / last PSN
Receipt bitmap	Received / missing
Retry / timer state	Expiry + attempts
Payload locator	WQE + byte offset

Implementation choices

Re-read host memory
Extra PCIe access

On-card replay buffer
Extra card storage

Out-of-Order Placement and Message Completion

One SEND message uses receive WQE #7.

Destination buffer B	0	1	hole	3	4
	Message-relative bitmap snapshot				
Receipt	1	1	0	1	1
Message end	0	0	0	0	1

CQE is held while receipt contains a hole.

Retry P2 fills the hole; all required data placement is complete.

Receive retry P2
Receipt = 11111 → Advance message completion frontier → Publish receive CQE #7

The last packet can arrive before the message is complete.

Ordering Across Operations and Queue Pairs

Same RC QP

RC, host memory, normal ordered MR, and a pre-posted RECV for ready.

QP 7: WRITE(data)

↓

QP 7: SEND(ready)

↓

Successful ready CQE
Data can be used

Different QPs

QP A: WRITE(data)

Independent connections

QP B: SEND(ready)

↓

Ready CQE may arrive
while data is incomplete

Within one QP: progress gates notification

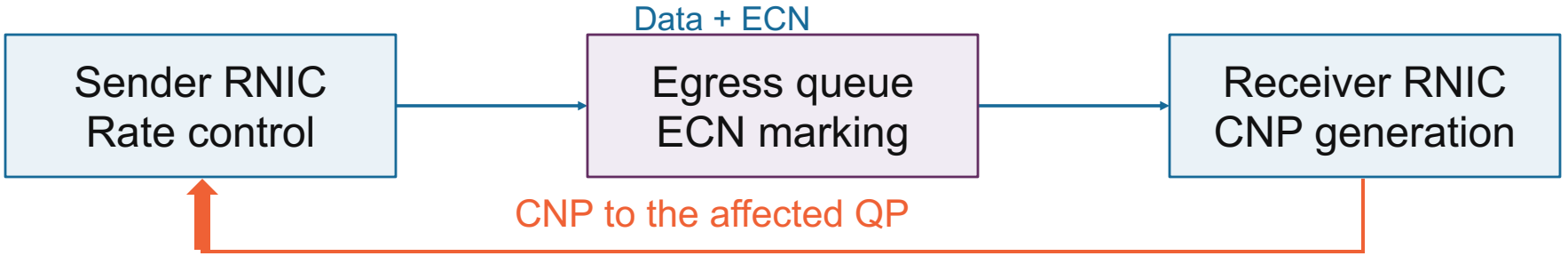
Prior work incomplete
Hold ready CQE

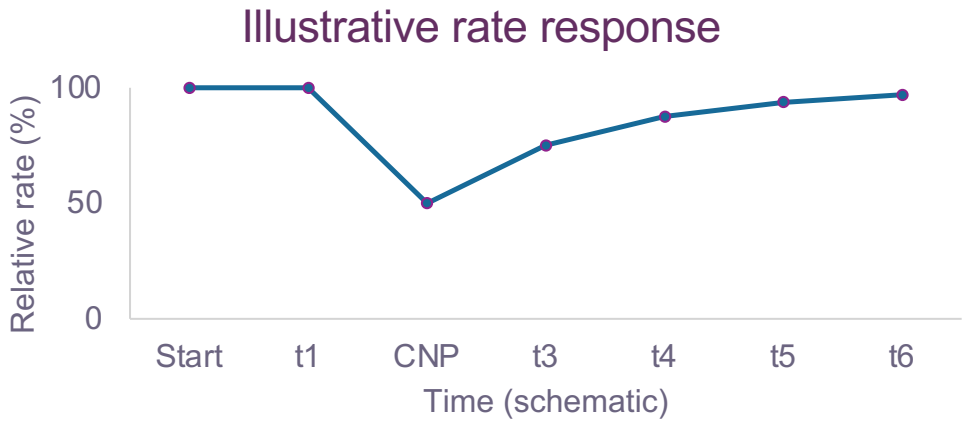
→

Prior work complete
Release ready CQE

DCQCN: From ECN Marks to Rate Feedback

Switch marks data; receiver reports; sender adjusts rate.



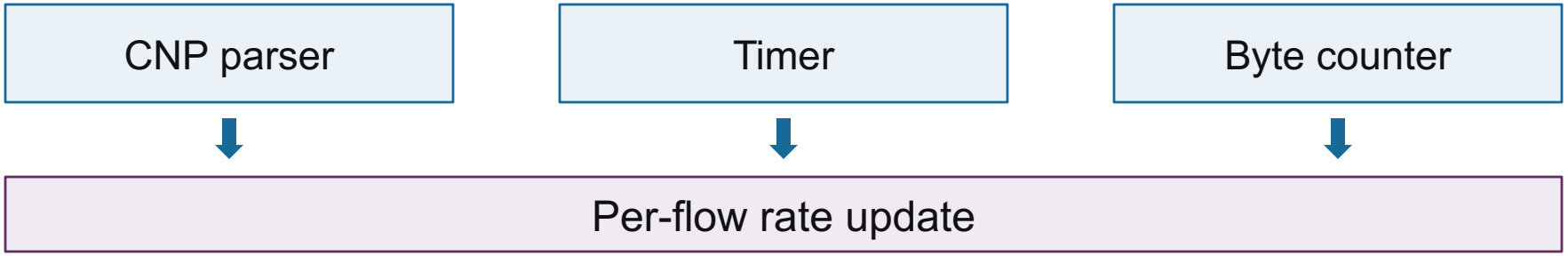


CNP reduces rate.
Timers and byte counters guide rate recovery.

CNP: congestion
ACK/NACK: delivery, recovery

Source: DCQCN §3.1, Figs. 5–7.

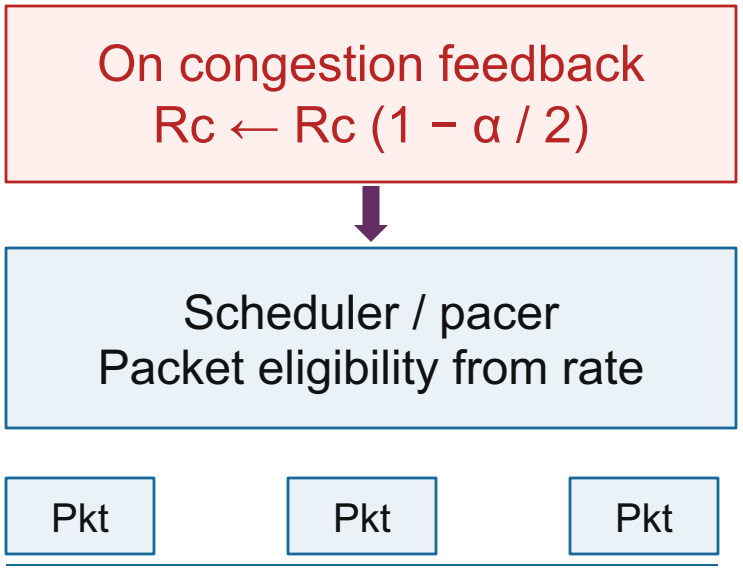
Congestion Control in the RNIC



Per-flow state	Meaning
Current rate Rc	Rate enforced now
Target rate Rt	Recovery target
Congestion estimate α	Smoothed feedback
Timer / counter	Recovery trigger

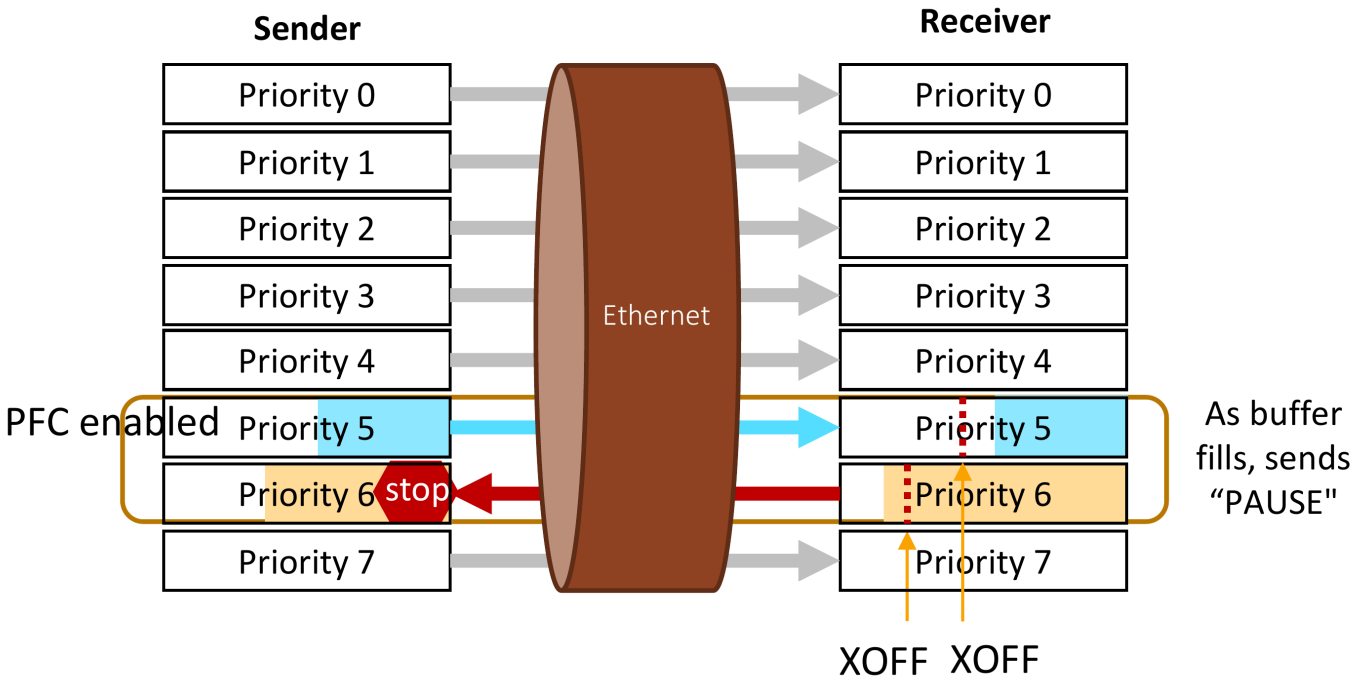
Lower rate → wider packet spacing

Eligibility interval ≈ packet bits / rate



PFC: Protecting the Next-Hop Buffer

PFC pauses a priority on one link.

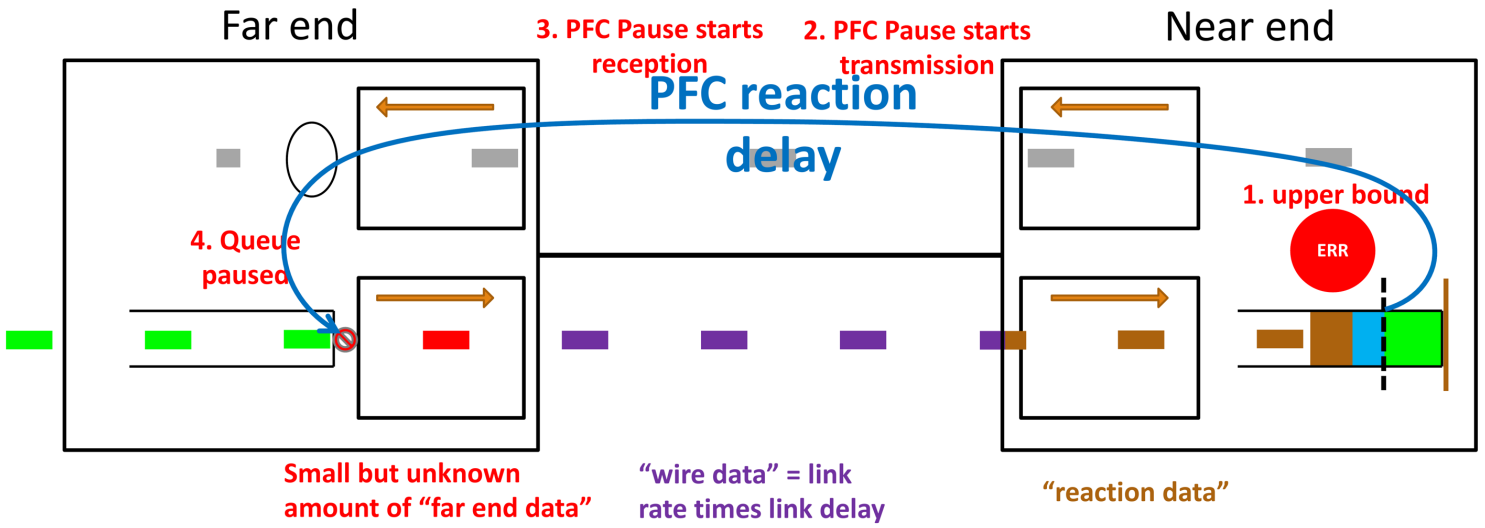


Occupancy counter → XOFF / XON comparison → PFC frame generator → Upstream MAC → Priority TX gate

Pause duration and XOFF / XON hysteresis govern pause and resume.

Source figure: IEEE 2023 Data Center Network webinar, PDF p.4 (right).

PFC Headroom and Backpressure



100 Gb/s × 2 µs = 25 KB of reaction data

Assumed reaction delay. Headroom also covers wire and committed data.

Flow A + Flow B Share one priority → PFC pauses the priority → Both flows stall Pause can spread

Source figure: IEEE Finn PFC Headroom, PDF p.18. 2 µs example is an assumed reaction-data term.

InfiniBand and RoCEv2 Packet Formats

Common: RDMA operations / QP / reliable transport

Native InfiniBand

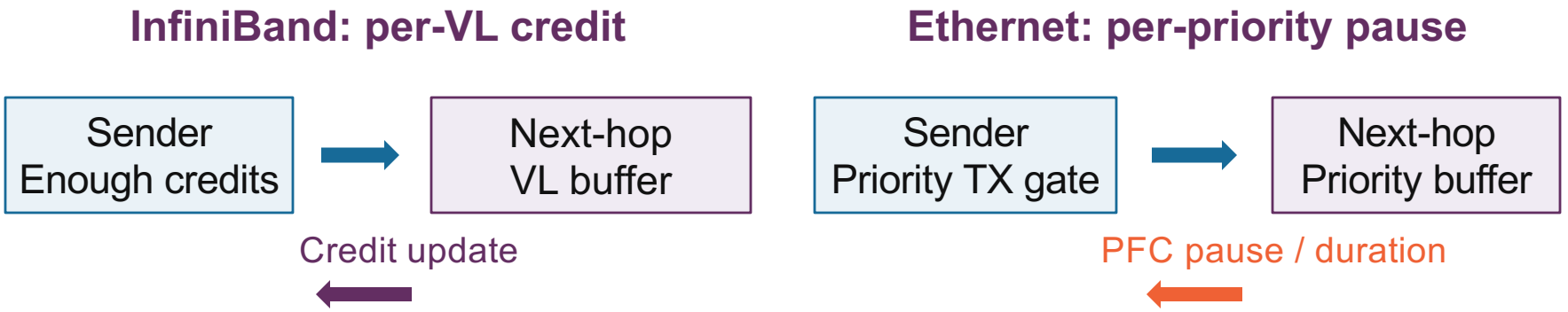
LRH	GRH (optional)	BTH	Operation header	Payload	ICRC	VCRC

RoCEv2 over Ethernet

Ethernet	IPv4 / IPv6	UDP	BTH	Operation header	Payload	ICRC	FCS

RoCEv2 uses IP routing and UDP encapsulation.
The RC transport supplies reliability and ordering.

InfiniBand vs Ethernet: Fabric Responsibilities



Dimension	Native InfiniBand	RoCEv2 over Ethernet
Forwarding	LID in subnet; GRH across subnets	Ethernet + IP; routing and ECMP
Link flow control	Per-VL credits	Per-priority PFC (lossless setup)
Congestion	IB congestion signaling	IP ECN + CNP DCQCN example
Management	Subnet Manager	Ethernet / IP control

Summary

Kernel Bypass and RDMA: Two Design Choices

	Kernel bypass + selective offload	RDMA
Interface	Sockets through a stack / runtime	Verbs + registered buffers
Protocol control	Software-defined CC, recovery, policies	Transport mechanisms executed by the RNIC
Offload boundary	Choose functions for each stack / workload	Transport + direct data placement
Workload fit	Diverse cloud services; per-service stacks	Storage, databases, AI; structured communication
Trade-off	Software freedom; CPU / runtime cost remains	Less protocol freedom; low host cost and a more uniform fast path

Software flexibility vs. a more uniform hardware transport path

Workload examples overlap. End-to-end performance depends on the deployment.

Scaling Out AI Networks

Lecture 3: Network Evolution
AI Clusters at 10K and 100K GPUs

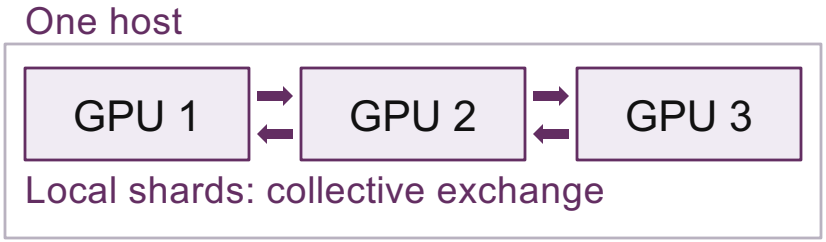
Training communication
Network design landscape
Alibaba HPN case study

THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

AI Communication Has Structure

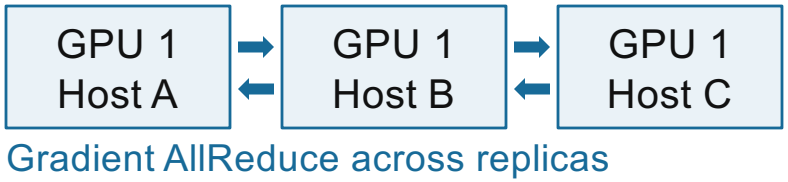
Training groups and communication phases repeat across iterations.

Tensor parallel



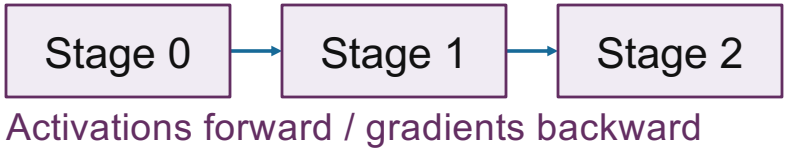
Fast local links for frequent exchanges

Data parallel



Map repeated groups to matching paths

Pipeline parallel



Match traffic to domain bandwidth

Expert all-to-all adds patterns that need broader connectivity.

Opportunity: co-design topology and collective communication.

AI Traffic Arrives in Bursts and Heavy Flows

Bursts reach NIC capacity

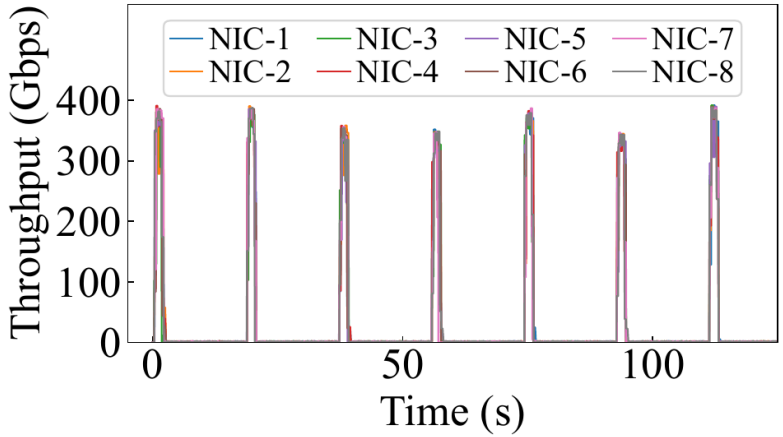


Figure 2: NIC egress traffic pattern during production model training.

A communication phase can raise demand across many GPUs.

Few concurrent connections

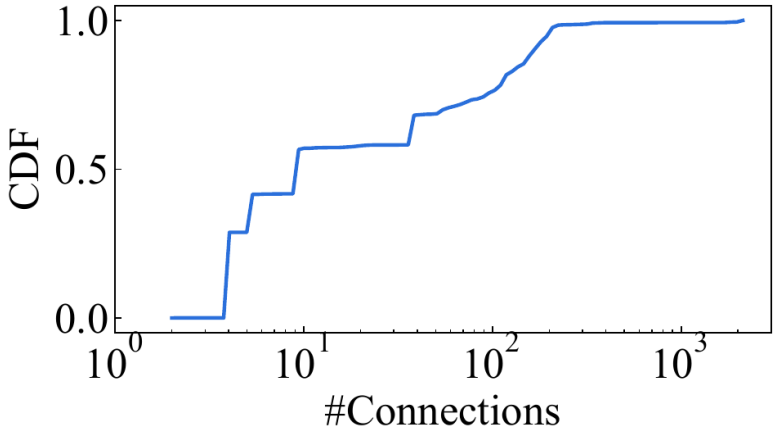


Figure 3: Number of connections per host.

A few heavy flows make hash-based balancing harder.

High, stable throughput needs burst capacity and balanced paths.

Source: SIGCOMM'24 HPN Figs. 2–3 / §2.2; measurements from production training.

Scale-Out Network Landscape

Design	Fabric	Organization	Path control and coordination
NVIDIA InfiniBand	IB	Rail-optimized fat-tree	Adaptive routing Collective / rail mapping
NVIDIA Spectrum-X	Ethernet RoCE	Clos / multiple planes	Per-packet adaptive routing SuperNIC handles out-of-order data
Alibaba HPN	Ethernet RoCE	Rail segments Dual planes	Library selects disjoint connections Schedules outstanding WQE bytes
Meta	Ethernet RoCE	Clos AI zones	Topology-aware placement ECMP / centralized traffic engineering
Tencent Astral	Ethernet RoCE	Rail aggregation domains	Per-flow ECMP Telemetry and runtime coordination

Fabric, topology, routing, and runtime choose different boundaries.

Scale-Out Network Landscape

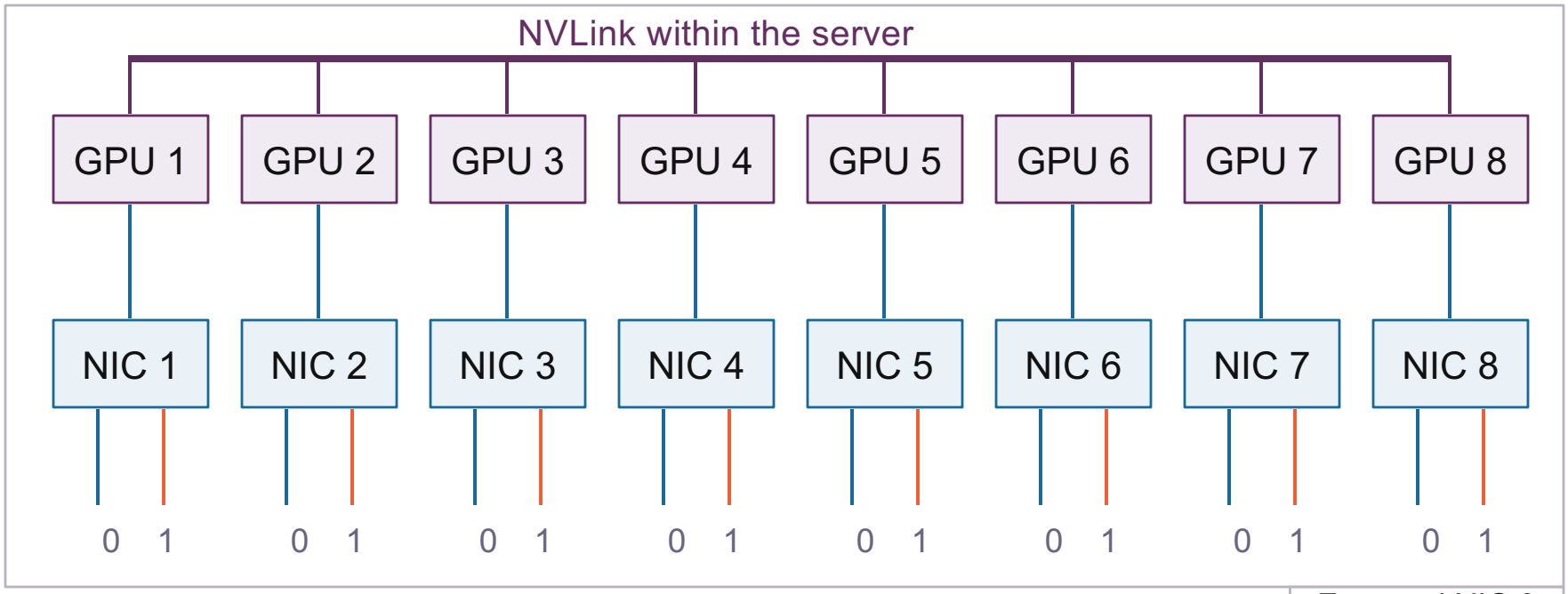
Design	Fabric	Organization	Path control and coordination
NVIDIA InfiniBand	IB	Rail-optimized fat-tree	Adaptive routing Collective / rail mapping
NVIDIA Spectrum-X	Ethernet RoCE	Rail-optimized dual-plane	Adaptive routing SuperNIC handles out-of-order data
Alibaba HPN	Ethernet RoCE	Rail segments Dual planes	Library selects disjoint connections Schedules outstanding WQE bytes
Meta	Ethernet RoCE	Clos AI zones	Topology-aware placement ECMP / centralized traffic engineering
Tencent Astral	Ethernet RoCE	Rail aggregation domains	Per-flow ECMP Telemetry and runtime coordination

Scale-out network design has yet to converge, so we turn to Alibaba HPN as a case study to explore the key design choices and trade-offs.

Fabric, topology, routing, and runtime choose different boundaries.

HPN: The Server–Network Interface

8 GPUs, 8 backend NICs, and 1 frontend NIC per server

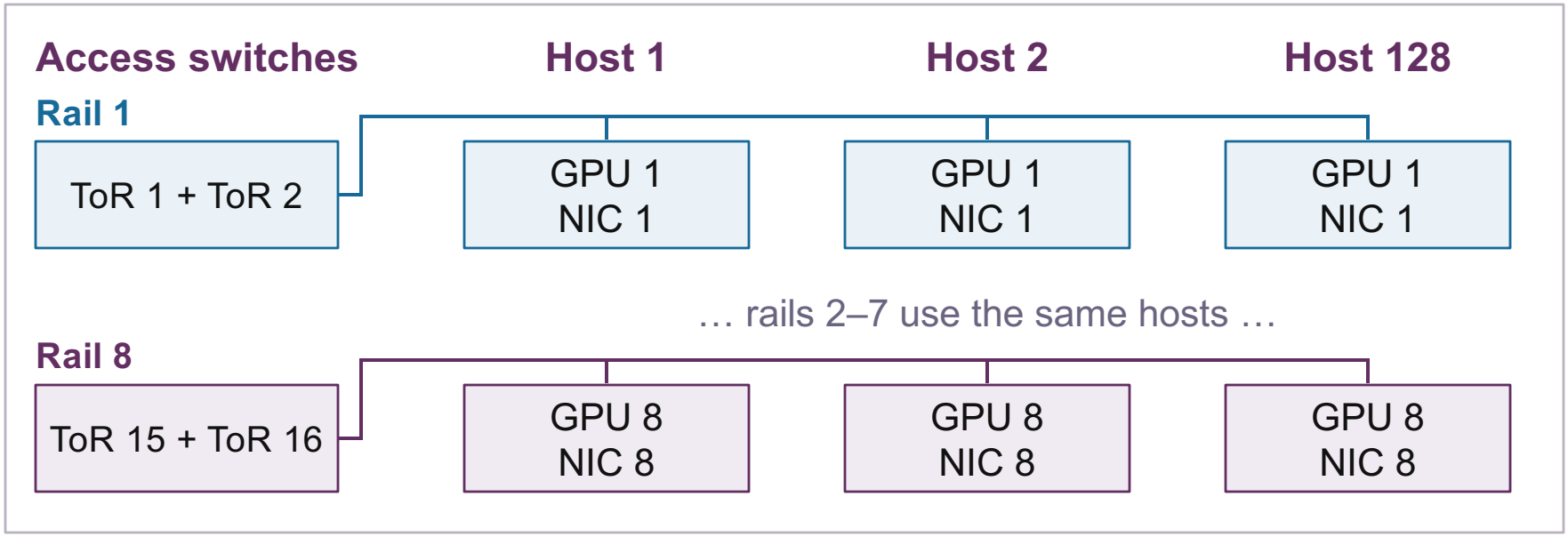


Backend **8 × (2 × 200 Gb/s) = 3.2 Tb/s per server**

Each backend NIC reaches two independent ToRs.

Rails and Segments

Rail: same-index GPU / NIC pairs across hosts share a ToR pair.



One segment = 128 active hosts × 8 GPUs = 1,024 GPUs

Eight rails cover the same 128 hosts, plus the associated ToR pairs.

Structured Communication on Rails

Example: 512 GPUs = 64 hosts × 8 GPUs, with TP=8, DP=64

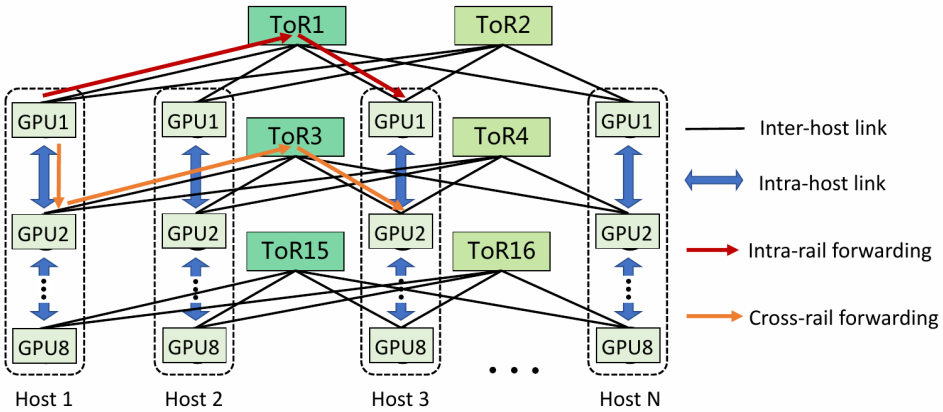


Figure 11: Rail-optimized network under dual-ToR.

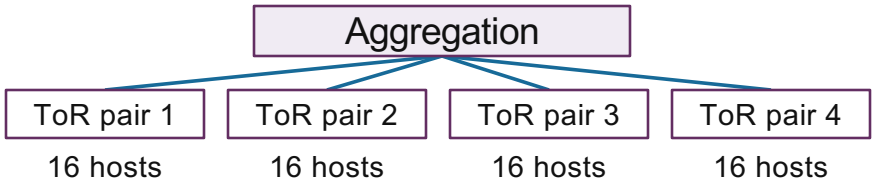
Known communication groups

TP peers use the fast intra-host interconnect.

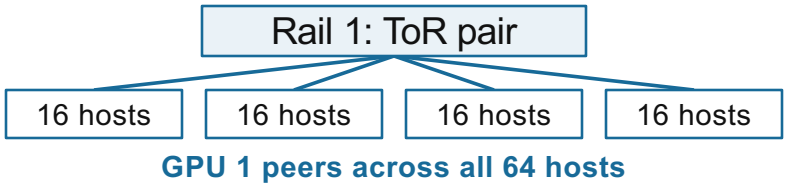
DP group 1: GPU 1 on all 64 hosts
Other indices form other groups.

Same-index DP traffic stays within its rail's ToR pair.

Traditional DCN: two switching tiers

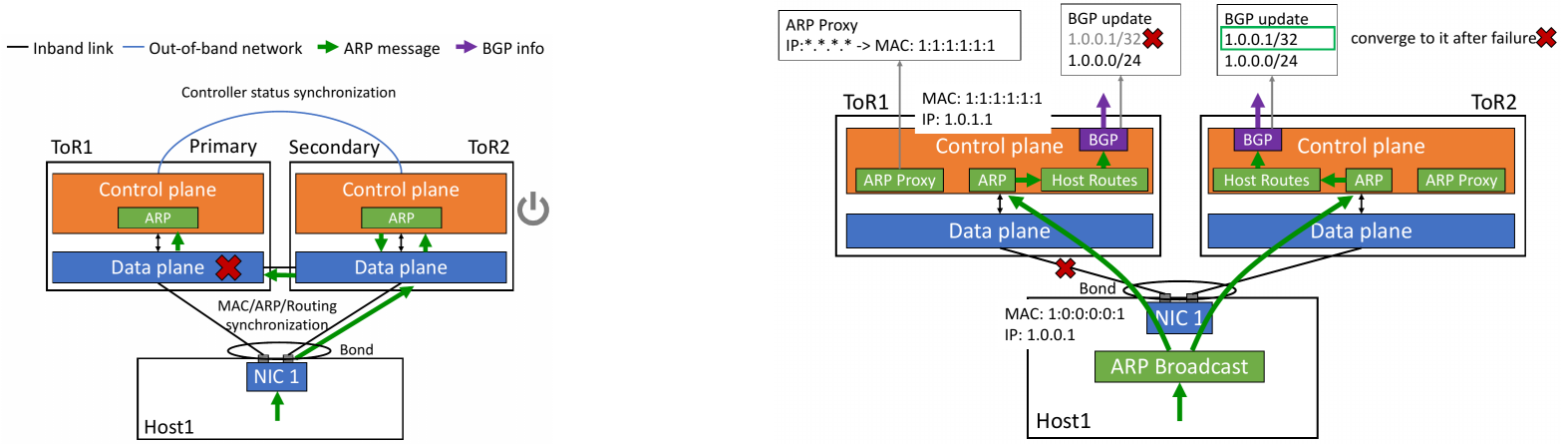


HPN: one switching tier per rail



Per ToR: 128 active 200G downlinks. Downlinks used per host: DCN 8 per ToR (16 hosts/pair), HPN 1 per rail ToR (128 hosts/pair).

Independent Dual-ToR Connectivity



(a) Stacked dual-ToR.

(b) Non-stacked dual-ToR.

Figure 8: Dual-ToR solutions. In stacked dual-ToR (a), the malfunction of ToR1's data plane would eventually trigger the offline of ToR2. In non-stacked dual-ToR (b), two ToRs run independently.

Two NIC ports, shared QP contexts

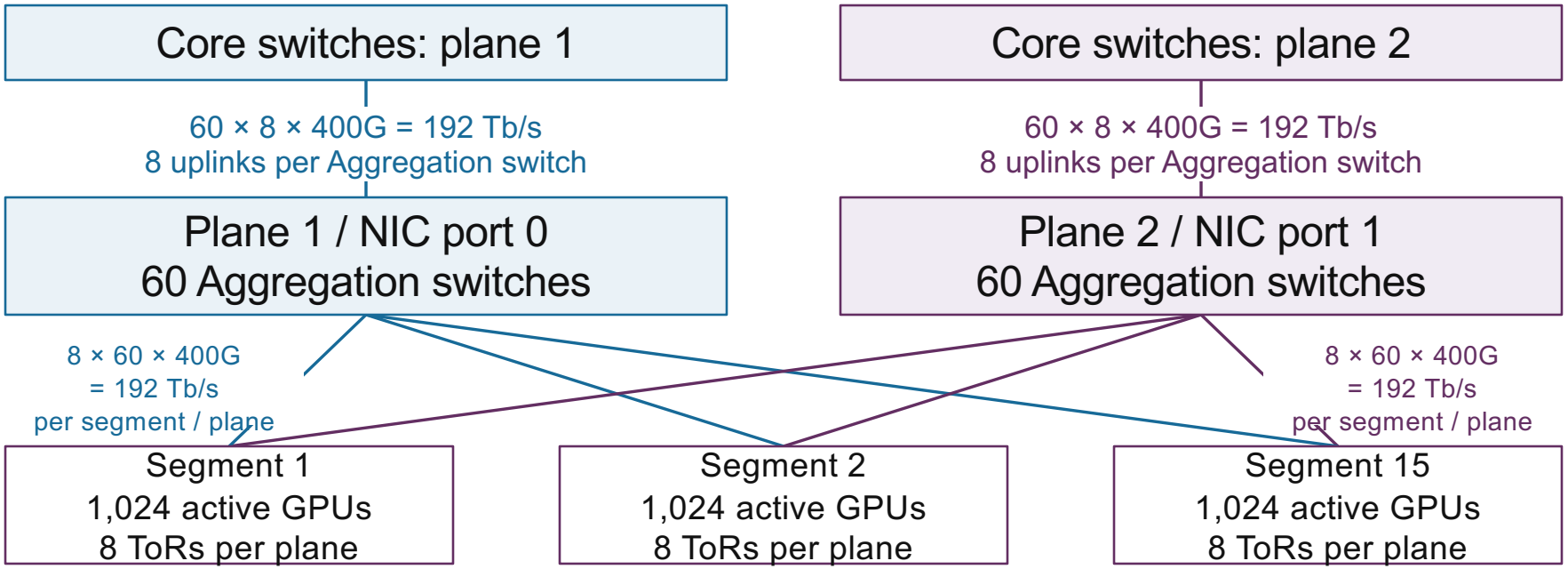
A failed NIC–ToR link redirects traffic through the other port.
Active RDMA QPs can survive the access-path change.

Independent ToRs remove the direct switch-state synchronization dependency.

Source: HPN Fig. 8 / §4. LACP, host-route withdrawal, and ARP details in notes.

Pods and Forwarding Planes

Plane: a separate set of forwarding paths and switch resources.

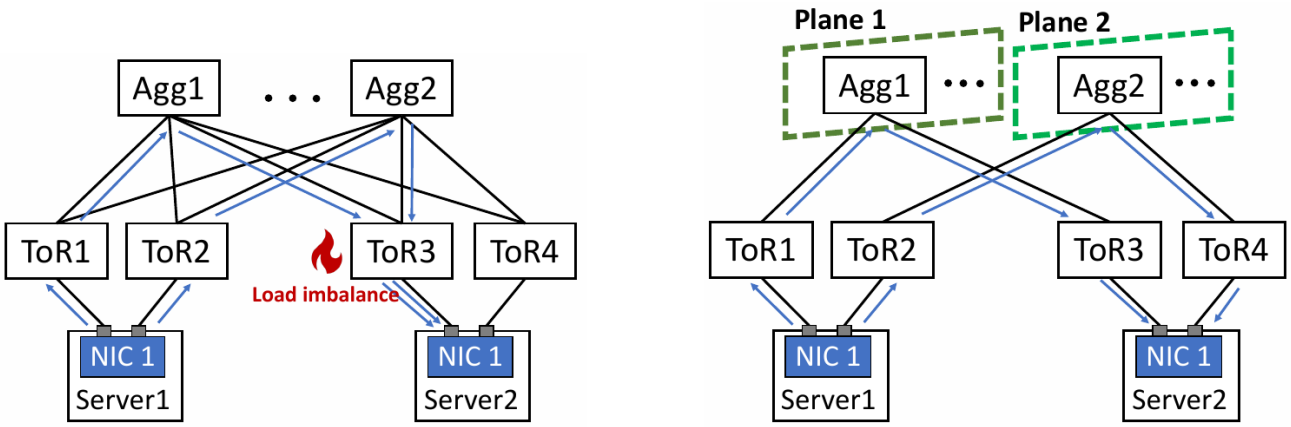


15 × 1,024 = 15,360 active GPUs per Pod

Per Aggregation switch: 120 × 400G down + 8 × 400G up = 51.2 Tb/s
120 downlinks = 15 segments × 8 ToRs. 48 Tb/s : 3.2 Tb/s = 15:1.

GPU injection: 1,024 × 200G = 204.8 Tb/s per segment / plane. ToR active down:up = 1.067:1.

Dual Planes Reduce Hash Polarization



(a) Typical Clos. (b) Dual-plane.

Figure 12: Tier2 network architecture.

NIC port 0 stays in plane 1. Port 1 stays in plane 2.

After the source ToR selects an uplink / aggregation switch, the remaining path inside the Pod is fixed.

Fewer hash stages reduce reconvergence during concurrent elephant flows.

Source: HPN Fig. 12 / §6.1.

Burst Waves and Path Collisions

A communication phase launches heavy transfers from several host pairs.

Host pair A	Host pair B	Host pair C	Host pair D

Hash collisions

Path 1	A	B	C	D
Path 2	idle			
Path 3	idle			
Path 4	idle			

Possible spread across paths

Path 1	A			
Path 2	B			
Path 3	C			
Path 4	D			

Disjoint connections give the library alternatives for each new message.

The burst still occurs. Path selection reduces spatial concentration.

The Collective Library Selects Paths

Simulate switch hashes → Choose disjoint source-port tuples → Establish RDMA connections

For each work item: choose the smallest outstanding-byte count

Connection	Outstanding before	Post 2 MB	1 MB WQE completes
A	8 MB	8 MB	8 MB
B (selected)	2 MB	4 MB	3 MB
C	5 MB	5 MB	5 MB

post WQE: add bytes CQE: subtract completed bytes

A slow-draining connection keeps more unfinished bytes.

Putting HPN Together

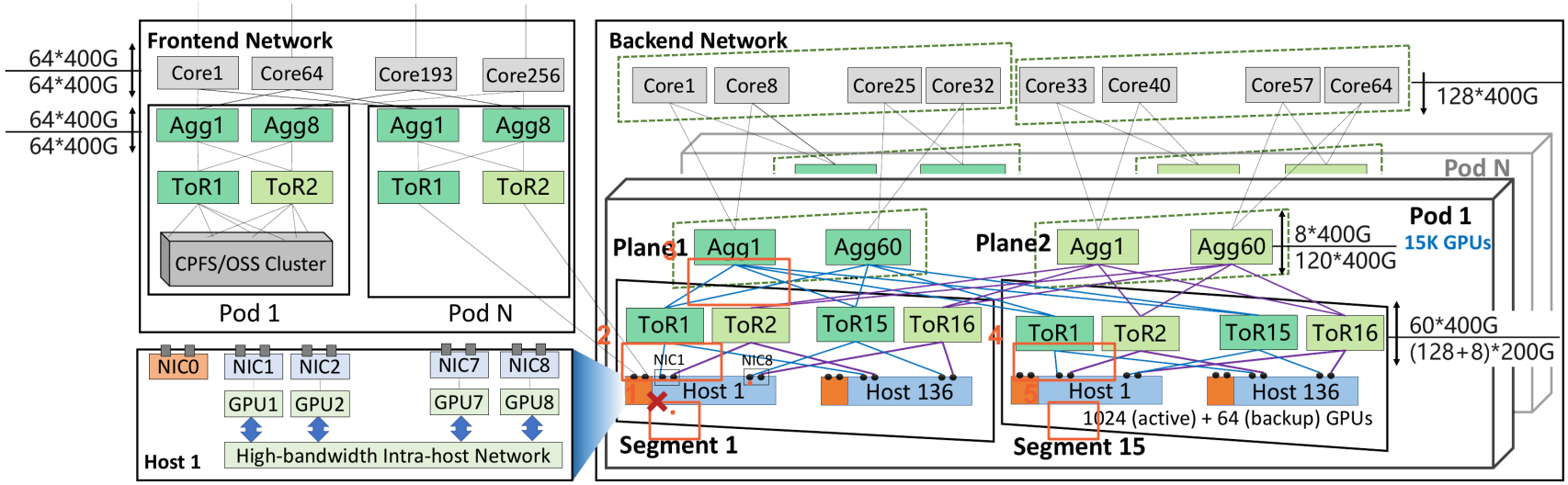


Figure 7: HPN overview. A solid parallelogram represents a segment (containing 1024 active GPUs and 64 backup GPUs). Two dotted parallelograms represent dual-plane. A cube contains an entire Pod (containing 15K GPUs).

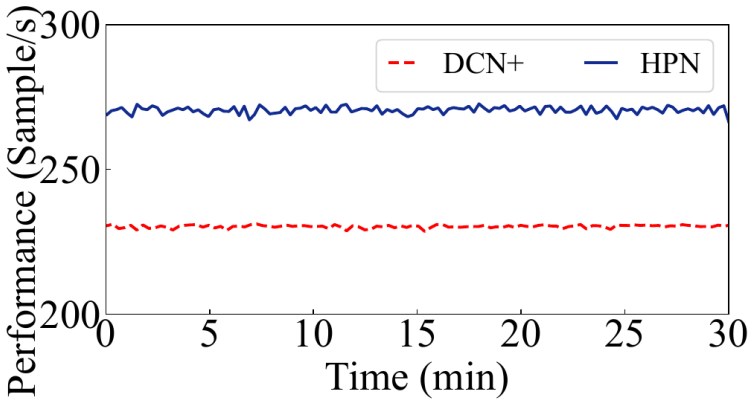
Access failure: the same NIC uses its other port and independent plane.

One cross-segment transfer

1 Host / NIC → 2 Source ToR → 3 Selected Agg → 4 Remote ToR → 5 Remote NIC

HPN: Evidence and Design Trade-offs

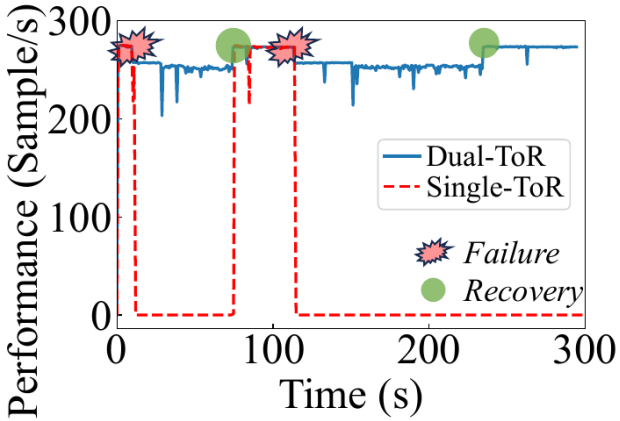
Training throughput



(a) End-to-end training performance.

2,300+ H800 GPUs, proprietary LLM
HPN throughput: +14.9% vs DCN+

Access-link failure



(a) Link failure.

256 GPUs, LLaMa7B
One link fails: dual-ToR loses 6.25%

Tier 2 retains any-to-any connectivity for MoE and cross-rail traffic.

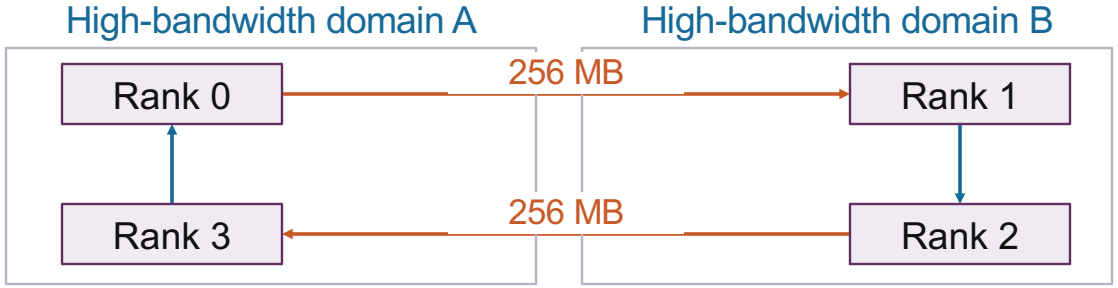
More port capacity in a Pod comes with less cross-Pod bandwidth.

Source: HPN Figs. 15(a) / 18(a).

Beyond 10K GPUs: Communication Locality

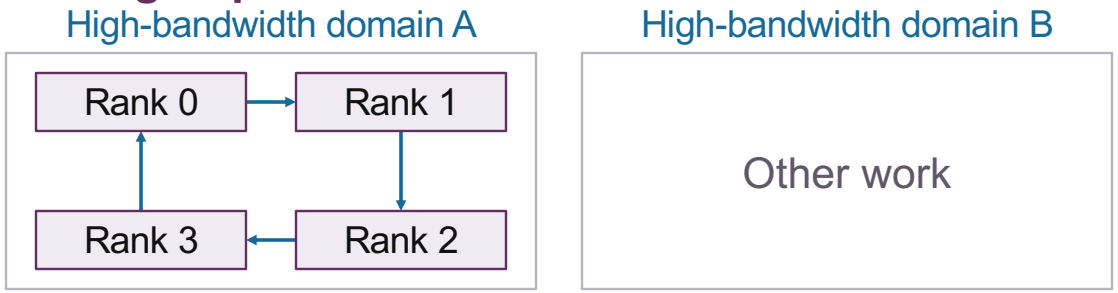
Example: one ring round, with 256 MB sent on each edge.

Spread group



2 edges cross domains
2 × 256 MB = 512 MB
of cross-domain traffic

Local group

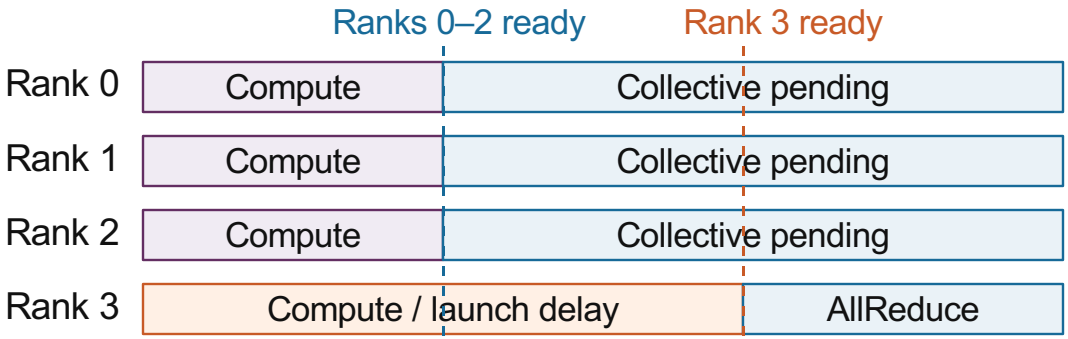


0 cross-domain bytes
Same ring and data volume
4 free GPU slots needed in A

Same 1,024 MB exchanged. Placement changes the cross-domain load.
Choose a domain with enough GPU slots and local bandwidth.

Beyond 10K GPUs: Diagnosing a Slow Collective

Example: Rank 3 enters a four-rank AllReduce late.



What the trace suggests

Rank 3 enters late.
Others may wait or make partial progress.
Check input, compute, and launch timing.

A long collective interval can include waiting for a late rank.



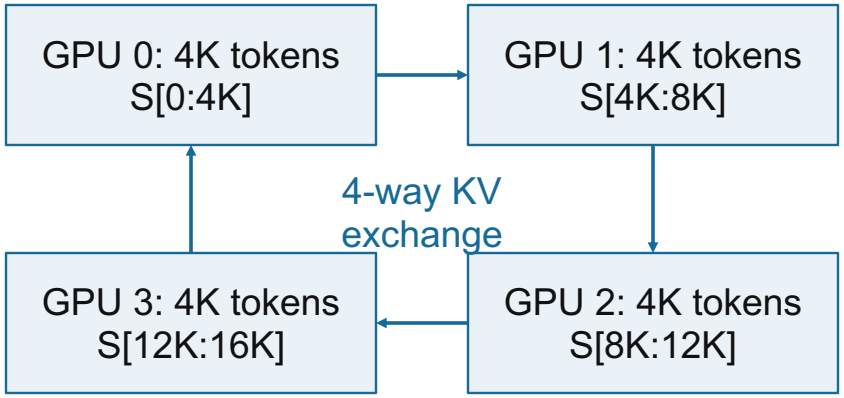
Evidence to correlate	Likely cause	Next action
Rank 3 enters late. Queues / errors look normal.	Input / compute / launch skew	Fix input, compute, or launch timing.
Ranks enter together. One path queues more.	Path congestion / load imbalance	Remap path or adjust send timing.
Link / CRC errors plus retry events	Link / device fault	Fail over / repair. Coordinate runtime recovery.

Re-measure collective time and useful training throughput after the fix.

Beyond 10K GPUs: Adapting Sequence Partitions

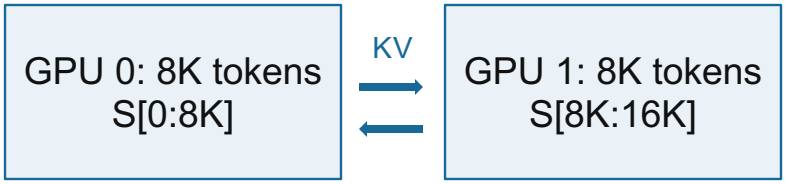
One 16K-token sequence. Assume capacity of 8K tokens per GPU.

Fixed four-GPU partition



16K / 4 = 4K tokens per GPU
4 GPUs exchange KV for this sequence

Two GPUs for this sequence



16K / 2 = 8K tokens per GPU
2 GPUs exchange KV for this sequence

Remaining GPUs handle another sequence

GPU 2	GPU 3
Other sequence	Other sequence

The same attention still covers all 16K tokens.
Choose the participants before choosing network paths.

Each selected GPU handles more tokens (4K to 8K).
Global gradient synchronization is still required.

Requirements for 100K-GPU Networks

Requirement	Mechanisms already seen
Capacity and locality	High-bandwidth domains Topology-aware rank / group placement
Path balance and bursts	Planes and disjoint paths Library scheduling, pacing, bounded in-flight data
Fault isolation and recovery	Independent access paths Fast failover with transport / runtime coordination
Observability and runtime	Correlate rank, QP, path, and port Validate recovery and reduce communication demand

Place work. Choose paths. Control when and how much to send.

100K GPUs require usable bandwidth for the communication groups that run.

Scaling Up Accelerator Networks

Lecture 3: Network Evolution
Tightly Coupled Accelerator Domains

Superpod landscape
Design requirements
EtherNode: endpoint, fabric, middleware



THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

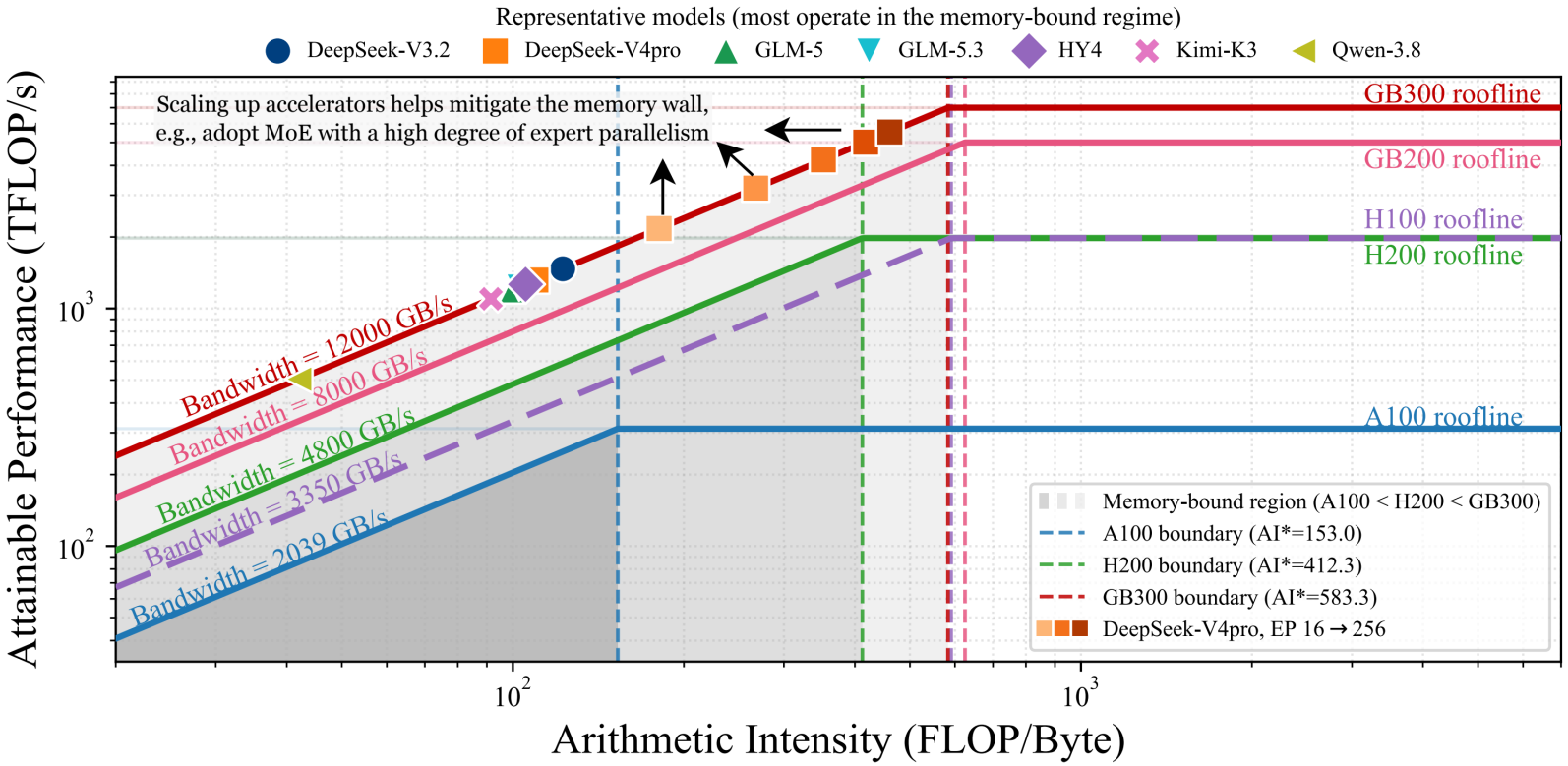
The Accelerator Memory Wall

Bandwidth wall

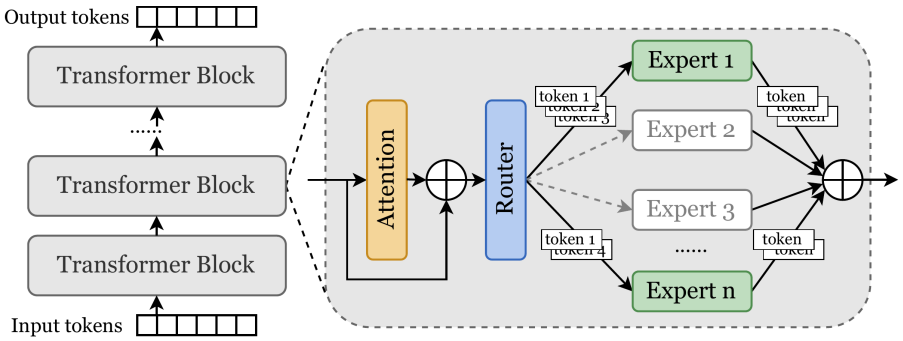
Compute outpaces HBM bandwidth.
Low AI limits attainable FLOPs.

Capacity wall

Larger weights and context states increase the memory footprint.



Aggregating HBM Resources



Capacity across HBM stacks

EP / CP partition weights and context states across devices.

Aggregate HBM bandwidth

Distribute memory traffic across stacks; exchange tokens and states over the interconnect.

EP degree	8	16	32	64	128	192
AI (FLOP/byte)	109.8	180.9	267.6	352.0	417.9	445.7
GPU utilization	18.8%	31.0%	45.9%	60.3%	66.2%	66.2%

Frequent dispatch, combine, and state exchange require a fast scale-up network.

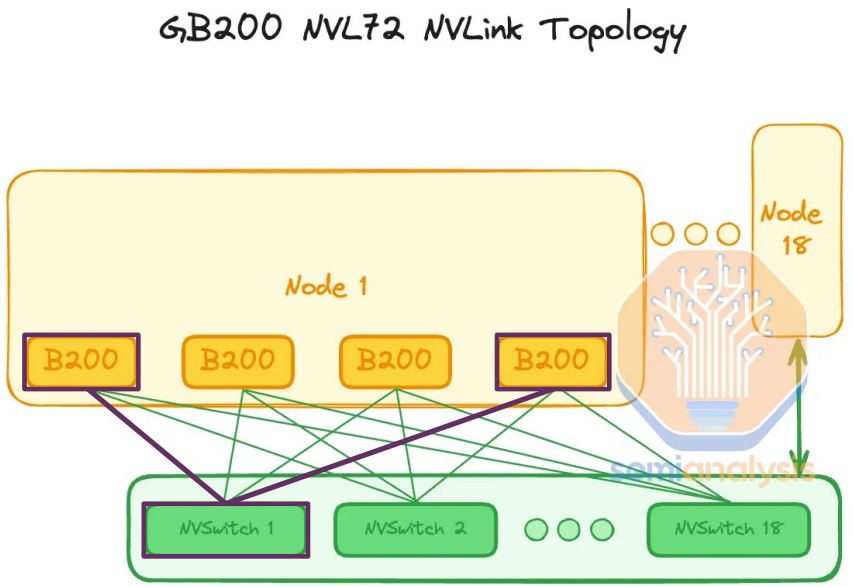
NVIDIA: A Rack-Scale NVLink Domain

72 GPUs share one rack-scale NVLink domain.

Rack and copper backplane **GPU → NVSwitch → GPU**



18 compute trays × 4 GPUs = 72 GPUs
9 switch trays × 2 ASICs = 18 NVSwitches



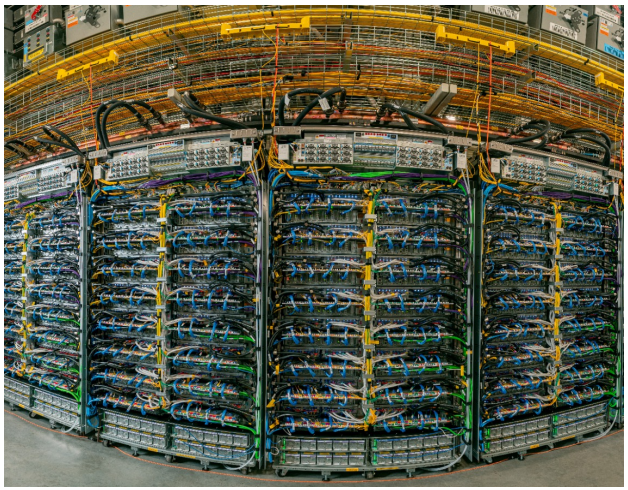
Example: two GPUs via NVSwitch 1

1.8 TB/s bidirectional aggregate per GPU.
IB / Ethernet connects separate NVLink domains.

Google: ICI Cubes and Optical Reconfiguration

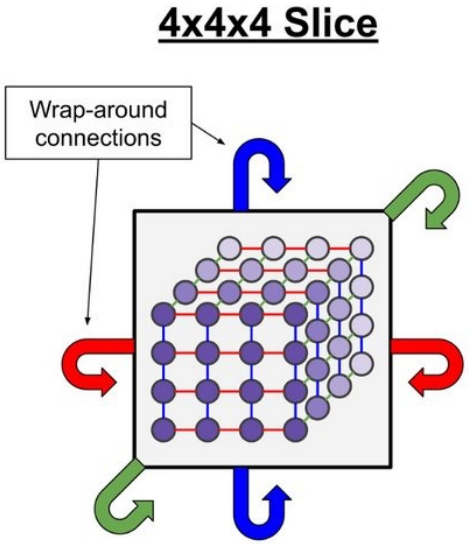
ICI links form a reconfigurable torus; DCN connects separate slices.

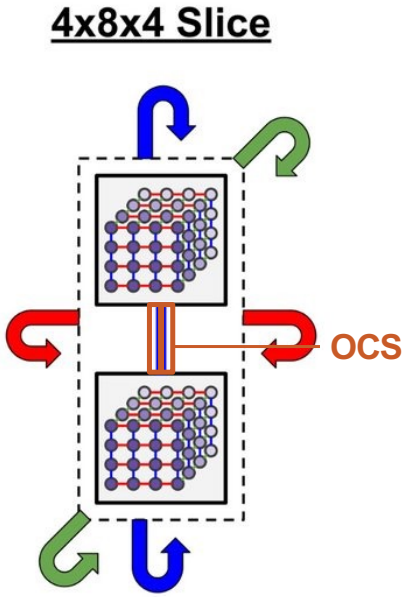
Ironwood physical system



64-chip cube
Six logical ICI neighbors per chip

One cube → two-cube slice





OCS reconnects cube boundaries into larger slices.
A large domain is not a single-hop full mesh.

Sources: Google Ironwood system architecture and AI-stack blog (Nov.2025); official slice illustration.

Huawei: A Multi-Rack UB Domain

CloudMatrix384 forms one multi-rack UB scale-up domain.

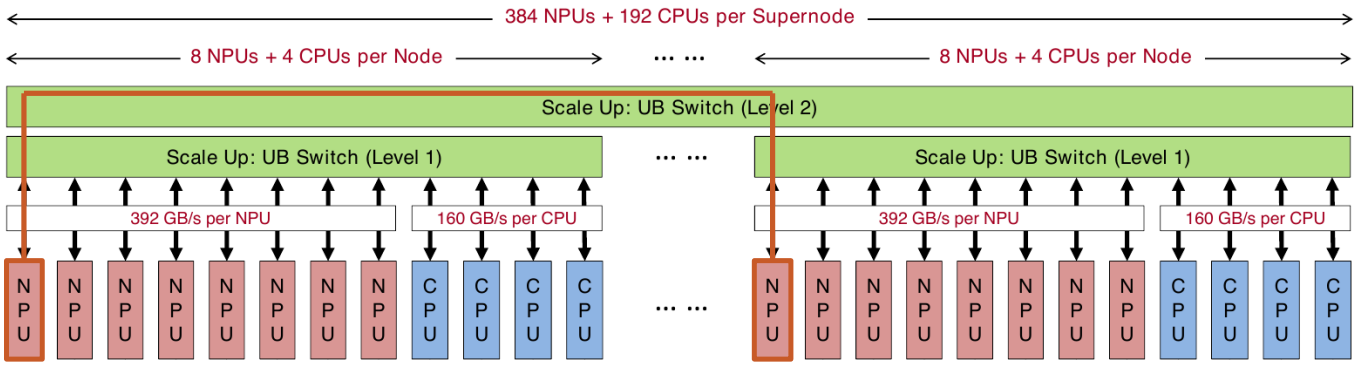


Atlas900A3 system render

384 NPU packages + 192 CPUs

48 servers span multiple racks.
UB uses two switching levels.
RoCE scale-out is a separate network.

UB path: NPU → L1 → L2 → L1 → NPU

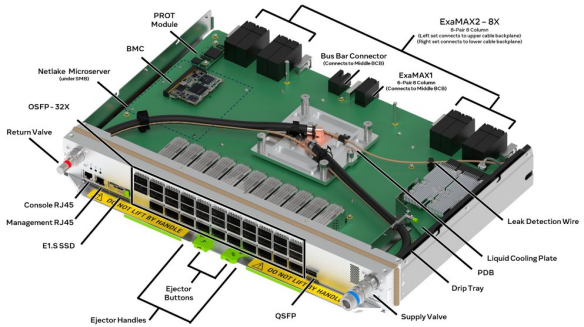


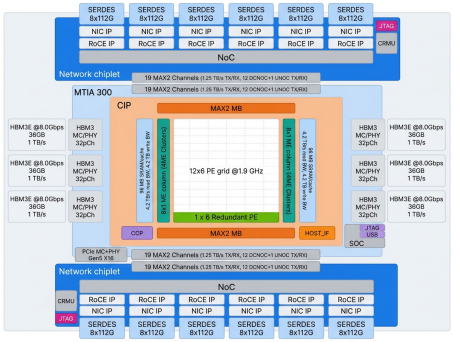
Sources: Serving LLMs on CloudMatrix384 v2, Fig.2 / §§3.2–3.3; Huawei Atlas900A3 product render.

Meta: Ethernet Scale-Up with Integrated NICs

MTIA300 pairs Ethernet switching with integrated network chiplets.

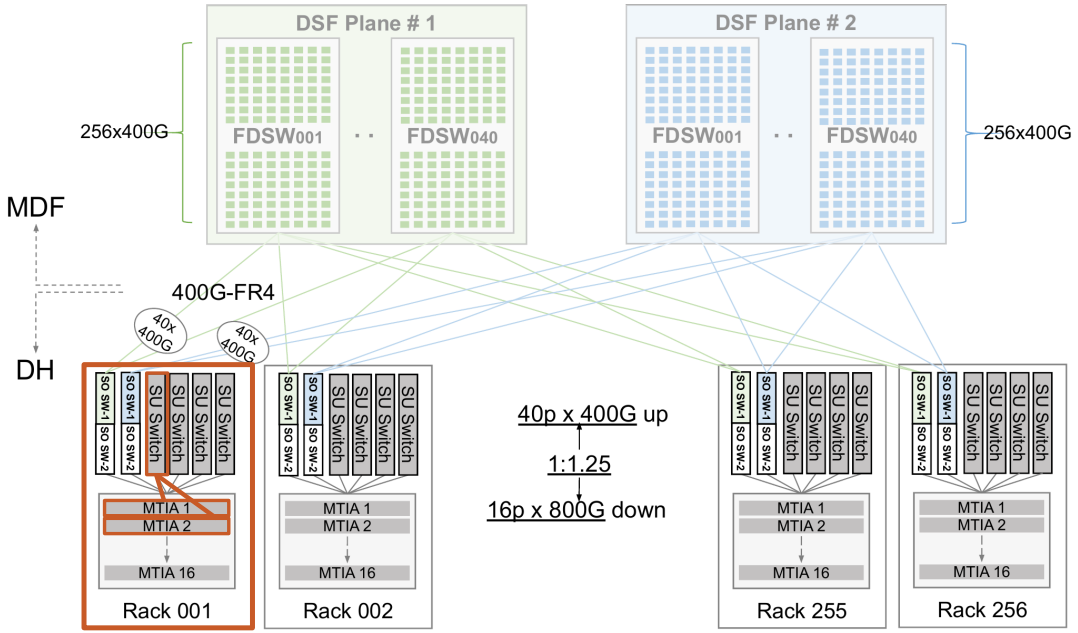
Tahan scale-up switch





Compute + network chiplets

Rack scale-up / DSF scale-out



MTIA 1 → rack SU switch → MTIA 2

One rack = 16 accelerator packages

Sources: Meta MTIA300 ISCA2026, Fig.7; Meta engineering blogs, Aug.24 and Oct.12,2026.

Alibaba: Modular ALink Supernodes

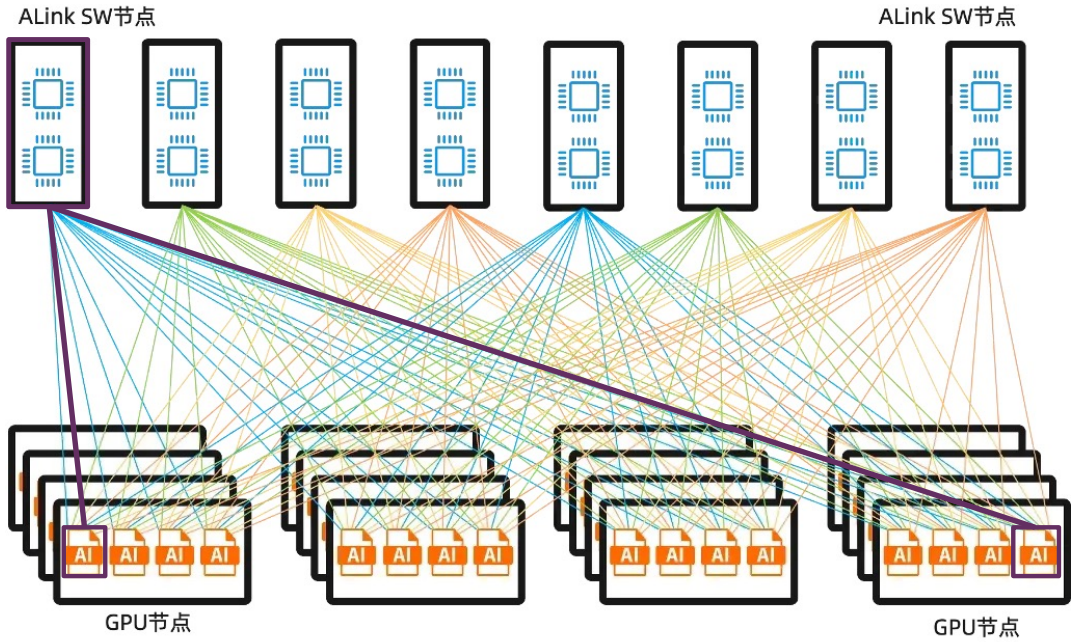
The 2025 cabinet disclosure contains two AL64 scale-up domains.

Front layout



Upper AL64 / lower AL64

One AL64: single-stage ALink



GPU node → ALink switch → GPU node

Source: Alibaba Cloud AL128 architecture disclosure, Oct.2025.

Design Requirements for a Scale-Up Domain

A scale-up domain needs bandwidth, reliable paths, and correct completion.

Three design questions

Endpoint: how many bytes can each device inject?

Fabric: which paths will carry those bytes?

Software: when may the receiver consume them?

EtherNode project goals

≥128 accelerators
≥200 GB/s per GPU (unidirectional)
Collective abstraction

16 MB transfer: serialization only

50 GB/s
320 µs

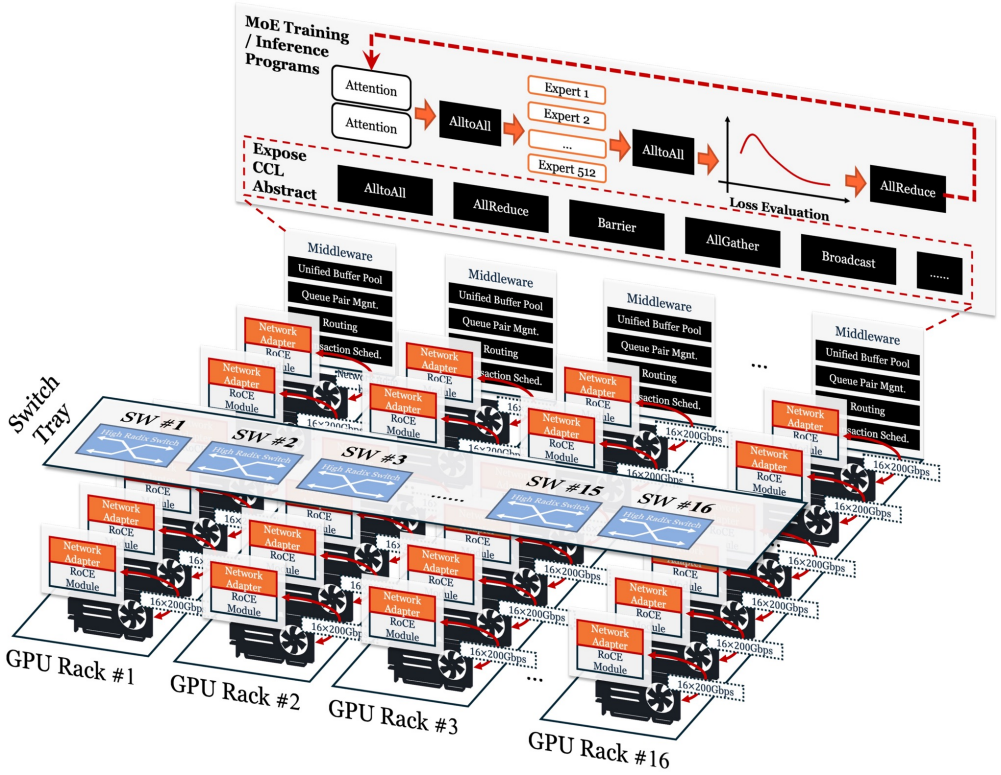


400 GB/s
40 µs

Beyond serialization: fixed overhead, queueing, and synchronization

EtherNode: Three Design Layers

One GPU-to-GPU transfer crosses three design layers.



Endpoint

Turn requests into reliable transfers on each path.

Fabric

Provide parallel, single-hop paths across the domain.

Middleware

Manage buffers and QPs; plan and execute collectives.

Endpoint Layer: From GPU Requests to Ethernet

GPU software manages transactions; packet engines deliver each path.

One collective request

↓

Transaction→ segments across paths

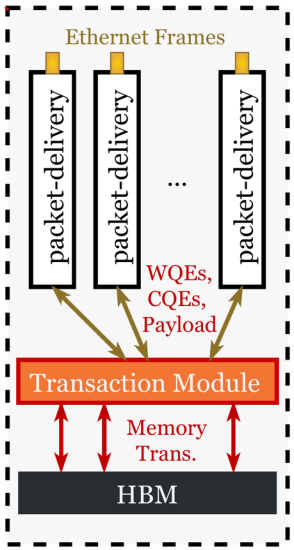
↓

WQEs / doorbells → selected QPs

↓

Packet-delivery engines: RoCE / MAC

Endpoint detail

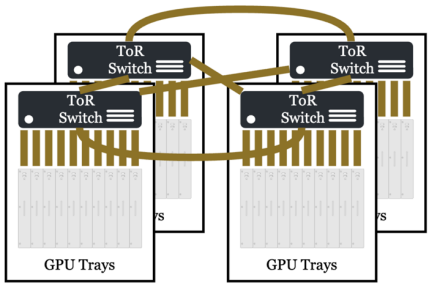


Payload: HBM ↔ packet engines ↔ Ethernet

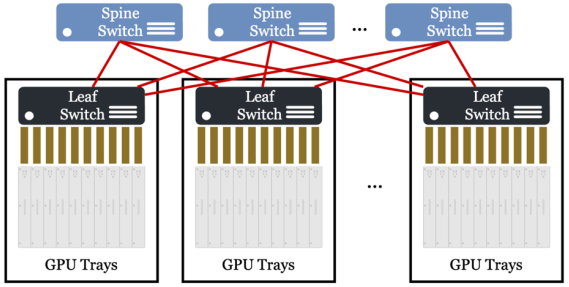
16 × 200 Gb/s = 3.2 Tb/s = 400 GB/s, unidirectional

Endpoint Layer: Optical I/O and Physical Reach

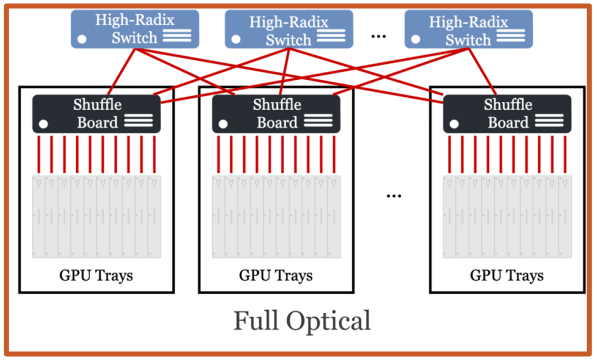
Move the electrical-to-optical boundary close to the endpoint.



Full Copper



Copper-Optical Hybrid



Full Optical

Full copper

Short reach
Dense rack cabling
Lower optics cost

Copper + optics

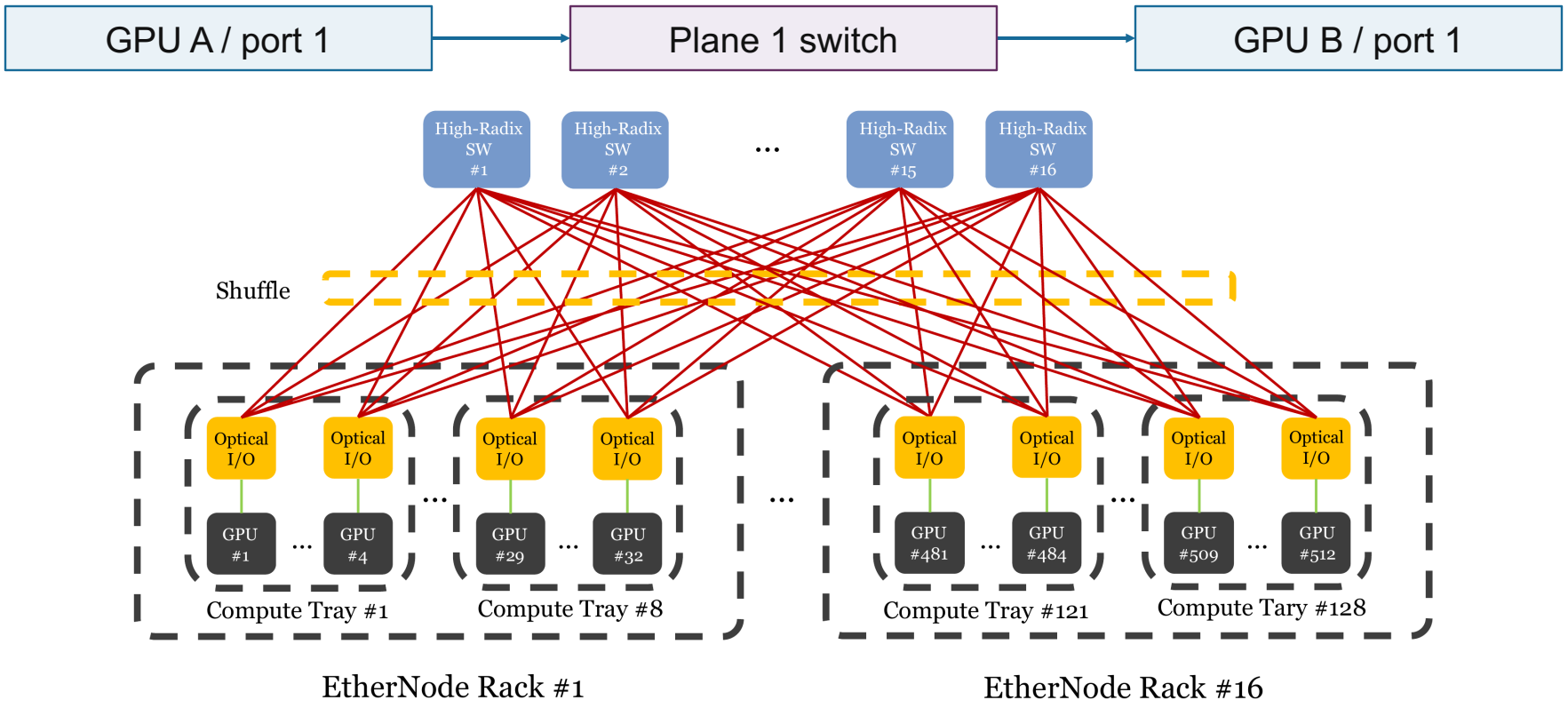
Longer reach
Additional switch tier
More complex paths

Full optical

Across racks, one tier
Higher optics cost / power
NPO or pluggable LPO

EtherNode chooses optical reach to preserve a flat fabric.

Fabric Layer: Sixteen Single-Hop Planes

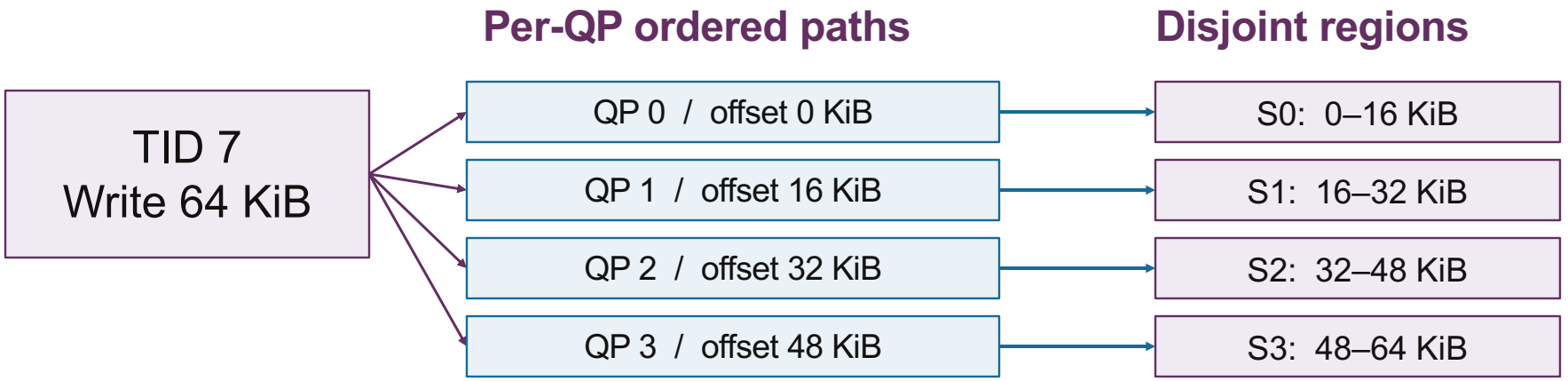


4 GPUs/tray × 8 trays/rack × 16 racks = 512 GPUs

Per plane: 512 × 200G endpoint-facing ports; no aggregation uplinks inside the domain.

Fabric Layer: Parallel Paths and Transaction Completion

Illustrative write: 64 KiB split over four of the sixteen planes.



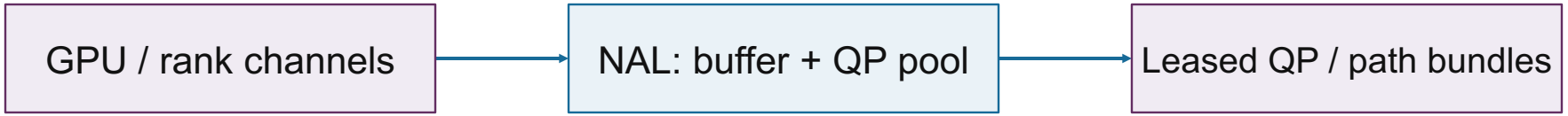
Arrives	Receipt: S0 S1 S2 S3
S2	0 0 1 0
S0	1 0 1 0
S3	1 0 1 1
S1	1 1 1 1

1111: transfers complete
Wait for preceding TIDs
Then commit TID 7

TIDs are per communication pair.
Example assumes earlier transactions have committed.

Middleware Layer: Buffers and QP Resources

The Network Abstraction Layer manages buffers, mappings, and QPs.



Static full-path provisioning

32 local GPUs × 511 peers × 16 paths

= 261,632 QP contexts

Available per rack
65,536 QPs

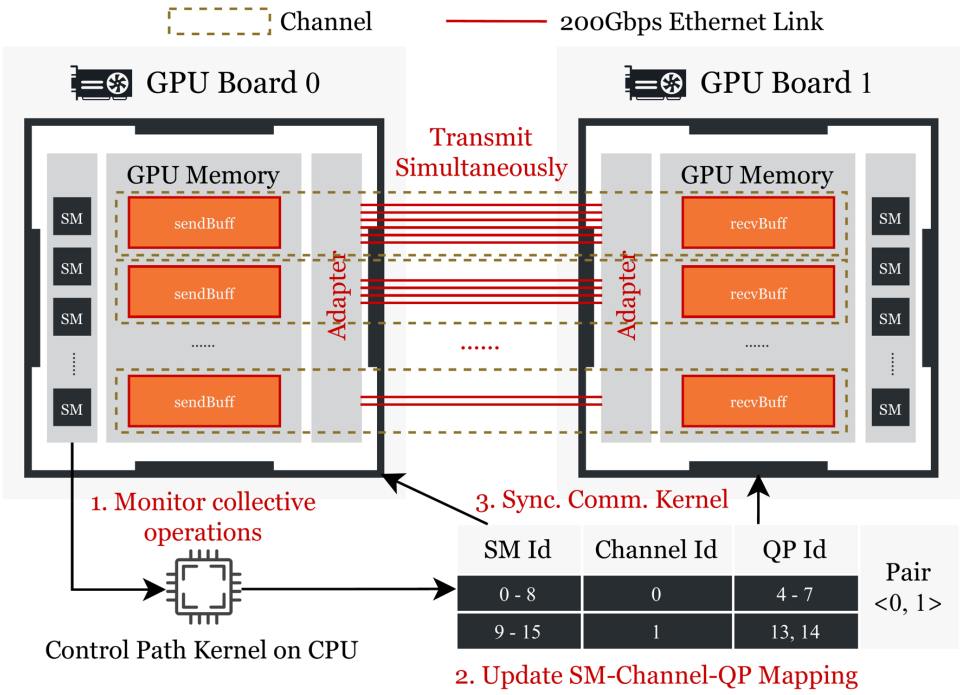
Allocate paths on demand. Reclaim only after outstanding work drains.

Unified buffer management is a software abstraction, not cache-coherent shared HBM.

Middleware Layer: Executing a Collective

An all-to-all phase becomes a plan of one-sided transactions.

Logical channels carried over switch planes



To	A	B	C	D
A	—	64 KiB	…	…
B	…	—	…	…
C	…	…	—	…
D	…	…	…	—

Resolve ranks, regions, lengths

↓

Bind SMs / channels / QPs

↓

Run GPU communication kernels

↓

Track received segments
Commit in order → phase complete

CPU daemon manages resources and refreshes mappings off the transfer path.

EtherNode: Summary of Design Choices

Scaling accelerator and memory domains over commodity Ethernet

Design constraint	Our design choice	Why this choice
Limited physical reach	**Full optical I/O**	Extend connectivity across racks while preserving a single-tier fabric
High bandwidth under a tight I/O-die budget	**Multi-plane, single-hop fabric with fixed per-QP paths**	Aggregate bandwidth while reducing packet-reordering buffer requirements
Cross-path ordering and finite connection resources	**Software transaction module and shared resource pools**	Coordinate completion and ordering. Allocate QPs and buffers on demand.

Optical reach adds cost and power. Software carries cross-path semantics and resource management.

A scale-up network combines high-bandwidth connectivity with explicit transfer semantics and resource management.